Software Design Document

for

OpenBench

Design of Release 1 against the Software Requirements Specification, version 0.4

Version 1.1, short edition

2026-11-07

Prepared by

Jai Nana Brian Lujan

Fotih Turopov Yerassyl Shnazbayev

CSDS 393 Software Engineering

Case Western Reserve University

Revision History

Name	Date	Reason for changes	Reviewed by	Version
J. Nana	2026-10-05	First complete draft: architecture, module specifications, runtime scenarios, data, API, crosscutting concerns, evolution, traceability.	Team inspection, Appendix C	1.0
J. Nana	2026-10-05	Short edition condensed from the full edition for submission; names, signatures, identifiers, and the inspection record unchanged.	Team inspection, Appendix C	1.0, short
J. Nana	2026-11-07	Aligned with SRS 0.4 (**RC-4**, **RC-5**).	None yet	1.1

How to read this document. Section 1 states the baseline and the principles; Section 2 gives the architecture; Section 3 allocates every responsibility to a module; Section 4 specifies each module with one template; Section 5 shows the transactions, the state machine, and the scenarios; Sections 6 and 7 give the schema, the API, and the crosscutting concerns; Section 8 says how the design evolves; Appendix A traces every requirement of the SRS to a module and a section. Capitalized words such as Request, Hold, and Loan have the meaning given in the glossary, Appendix B; code identifiers are set in `monospace`; module names in **bold sans-serif**. A longer edition of this design with the operation tables, worked examples, every scenario, and the full endpoint list is kept in the project repository; identifiers are the same in both.

Contents

1 Introduction **2**

2 Architecture **3**

3 Problem Decomposition **6**

4 Module Specifications **8**

5 Runtime View **19**

6 Data Design **24**

7 Interfaces and Crosscutting Concerns **26**

8 Maintenance and Evolution **29**

9 References **31**

A Traceability **31**

B Glossary **33**

C Inspection Report **34**

List of Figures

1 Modules and their dependencies (UML component diagram) . . . 4
2 Deployment (UML deployment diagram) . . . 5
3 Classes of **Allocation** (class diagram) . . . 11
4 Classes of **Ledger** (class diagram) . . . 14
5 Classes of **Requests** (class diagram) . . . 15
6 Request state machine (UML state machine diagram) . . . 20
7 S1, submit a Request (UC-4), sequence diagram . . . 21
8 S2, the last Required approval allocates the Request (UC-6), sequence diagram . . . 22
9 S6, Release and waitlist fill (UC-9, UC-10), sequence diagram . . . 23
10 Database schema, core modules (entity-relationship diagram) . . . 24

1 Introduction

1.1 Purpose and Scope

This document specifies the design of Release 1 of OpenBench, the internal marketplace in which a Manager posts one Request for people, Exclusive resources, and Metered capacity, and the system allocates each Request Line under the policy of its Allocation Class and settles it in credits.

The design covers every Release 1 requirement of the SRS. It divides the system into nine back-end modules and a web client (Section 2) and specifies each module down to Java signatures, owned tables, invariants, and failure behavior (Section 4). The six Stretch requirements are designed only as extension points (Section 8).

1.2 Requirements Baseline and Changes

The baseline is the OpenBench Software Requirements Specification, version 0.4 of 2026-11-07. Identifiers are those of its short edition: 41 Release 1 and 6 Stretch functional requirements in dot notation, twelve use cases, and the CO, UI, SI, CI, PE, SE, QA, UD, AS, and DE identifiers. Version 0.4 differs from version 0.2, the edition that was graded, by the five requirements changes below (the last two in one row).

Change	What changed, and why	Where it lands
RC-1 One Loan at a time	A Member with a Loan overlapping any day of a Matched line's window is not a Candidate for that line; the Shortlist is ordered by fewest person-days lent in the current Period, ties by name (**Profile.Availability**, **Matched.Shortlist.Build**). Grader feedback.	`MatchedAllocator` (Section 4.5); the `loan` constraint (Section 6).
RC-2 Inspection participation	The inspection record of the SRS states that all four team members read and inspected it. Grader feedback.	No design effect.
RC-3 Test database	Automated tests run on PostgreSQL 16 in a container started by the test suite (Testcontainers), with the same migrations as production; the in-memory H2 database is dropped (**CO-2**, **SI-1**). Design decision (ADR-0006).	Sections 6 and 7.
RC-4, **RC-5**	Optional approvals never hold a Request in Pending; a Refund's lead time runs to the start of the occurrence.	**Lifecycle.Evaluate**, **Release.Refund**; S1 to S4.

1.3 Design Principles

Simplicity first. The team's standing rule is the simplest architecture and program that meets the SRS: one process, one database, direct method calls, no message broker. **Modularity, low coupling, and high cohesion.** Each of the nine modules hides its entities and tables behind its application services; dependencies point down a fixed layer order with no cycles, and each module owns one cluster of the vocabulary: **Allocation** decides all capacity and **Ledger** all credit arithmetic, and neither calls the other. **Separation of concerns.** What state a Request is in (**Requests**), whether capacity exists (**Allocation**), and what it costs (**Ledger**) are answered by different modules. **Correctness enforced in the database.** Invariants that must survive concurrent users are database constraints, row locks, or grants, not only application checks (ADR-0006). **Uniformity of patterns.** Every module has the same four inner layers, every user or job action is one transactional service method, every module is specified with the same template, and every error is a problem response with a stable code.

1.4 Notation and Conventions

The figures are UML 2 diagrams drawn with PlantUML from sources kept with this document; color carries no meaning, so every figure reads in grayscale. Three elements go beyond plain boxes and arrows; messages in sequence diagrams are numbered and named after the operations of Section 4. **Interface lollipop and socket:** in the component diagram the port `ReductionGuard` is drawn as a provided interface (a ball, or lollipop) on **Allocation** and a required interface (a socket) on **Inventory**; the call at run time goes from Inventory to Allocation, but the compile-time dependency goes the other way. **Dashed arrows labeled read-only:** the dashed arrows from **Dashboard** stand for SQL reads through views over other modules' tables, where a solid arrow is a call to a module's application service. **`alt` and `opt` frames:** in sequence diagrams an `alt` frame holds alternative branches, each with its guard in brackets, and failure branches are always drawn as an `alt` branch.

Requirement identifiers are set in bold: functional requirements keep their dot notation, for example **Lifecycle.States**, and other identifiers print with the numbers the SRS gives them, for example **QA-1**. Signatures are Java 17; every application-service method runs in one transaction unless marked *(read-only)*. All windows, Price Bands, and Periods are in the company's one time zone.

2 Architecture

2.1 Context

The five actors of the SRS are roles, not separate populations: every actor is a Member, and the Administrator may give one Member several roles, for example a Manager who is also the Capacity Planner. The Clock stands for the work the server does on its own when time passes, which runs inside the same process (**CO-5**). Release 1 has no external integrations (**LI-3**). The browser is the only client, and every flow is a REST/JSON call over HTTPS (**SI-2**, **CI-1**).

2.2 Architectural Style and the Dependency Rule

D-A Modular monolith (ADR-0005). OpenBench is one Spring Boot 3 application (Java 17, built with Maven), one PostgreSQL 16 database, and one React single-page application served by the same process. The back end has nine modules, one Java package each under `edu.cwru.openbench.<module>`: `members`, `inventory`, `settings`, `notifications`, `allocation`, `ledger`, `requests`, `dashboard`, and `jobs`. The web client is the tenth component. Modules collaborate by calling each other's application services (Spring beans) directly, inside the caller's transaction. There are no events between modules and no message broker.

The style follows from the two requirements that cannot bend: Holds commit and Charges and Earnings post in one transaction (**CO-4**, **QA-2**), and concurrent submissions never double-allocate or oversell (**QA-1**). With direct calls in one process, the calling service's transaction spans **Requests**, **Allocation**, and **Ledger**, so atomicity needs no compensation logic.

D-B The dependency rule keeps the modules apart inside one deployable:

(1) A module's only public surface is its application service classes and the DTOs they take and return. Entities and repositories are package-private.
(2) A module writes only its own tables (the sole-writer rule). Foreign keys may point into another module's tables as read-only references.
(3) Dependencies point downward in the layer order of D-C, with no cycles.
(4) The one upward hook is the port `ReductionGuard`: **Inventory** declares it and **Allocation** implements it, so Inventory can refuse a capacity reduction below future usage without depending on Allocation.
(5) **Dashboard** is read-only. It may query any table through SQL views, and it writes nothing.

One ArchUnit test class checks rules (1) and (3) and fails the build on a violation.

D-C Layers, top to bottom: the web client; then **Requests**, **Dashboard**, and **Jobs**; then **Allocation** and **Ledger**; then **Inventory**, **Members**, **Settings**, and **Notifications**. Inside every module the same four layers repeat: controller (REST), application service, domain (entities and pure rules), and repository (Spring Data JPA). Figure 1 shows every module and dependency. Three packages are shared infrastructure, not modules: `security` (the Spring Security configuration, which loads users from **Members**), `common` (the error types and their mapping to problem responses, the value types `Credits`, `DateRange`, and `TimeWindow`, and the typed ids), and `config`.

2.3 Module Map

The modules are listed bottom-up, in layer order; Section 4 specifies each in full.

Module	Purpose	Owns (tables)	Depends on
Members (4.1)	Members, Teams with exactly one Manager each, roles, the skill list, skills and availability ranges, the login identity	`team`, `member`, `member_role`, `skill`, `member_skill`, `availability_range`	nothing
Inventory (4.2)	Resource Types as data and Resources (instances, units, owner); Pool Capacity computed, never stored; declares the port `ReductionGuard`	`resource_type`, `resource_type_attribute`, `resource`, `resource_attribute_value`	**Members**
Settings (4.3)	Every Capacity Planner setting as data, with a change log; other modules read one typed snapshot, `PlannerSettings`	`period`, `grant_plan`, `type_pricing`, `price_band`, `refund_step`, `quota`, `planner_setting`, `setting_change`	**Inventory**, **Members**
Notifications (4.4)	In-app notifications written in the transaction of the change they report; `Delivery` with `InAppDelivery` now	`notification`	**Members**

continues on next page

Module map (continued)

Module	Purpose	Owns (tables)	Depends on
Allocation (4.5)	Whether capacity exists, and who has it: the `Allocator` interface, the registry, the three allocators, Holds, the Waitlist, Shortlists, approvals, Loans, the type lock	`exclusive_occurrence`, `metered_allocation`, `waitlist_entry`, `candidate`, `approval`, `loan`	**Inventory**, **Members**, **Settings**, **Notifications**
Ledger (4.6)	Everything about credits: Accounts, append-only Transactions, Statements, affordability, rollover, and the pure `Pricing`	`account`, `transaction`, `held_credit`, `period_close`	**Settings**, **Members**, **Notifications**
Requests (4.7)	The Request from Draft to a final state: Drafts, Suggestions, submission, the state machine driven by `RequestLifecycleService`, history, Release	`request`, `request_line`, `suggestion`, `request_event`	**Allocation**, **Ledger**, **Inventory**, **Members**, **Settings**, **Notifications**
Dashboard (4.8)	Showback figures scoped to the viewer's roles	no tables; views `v_type_hours`, `v_team_credits`, `v_waiting`	read-only SQL; **Members**
Jobs (4.9)	`DueWorkJob` every 60 seconds and `PeriodRolloverJob` daily and at start-up; the `Clock` bean	nothing	**Requests**, **Ledger**, **Settings**
Web client (4.10)	The React single-page application, one route per screen of the SRS; never computes a Quote	nothing	the REST API only (**CO-3**)

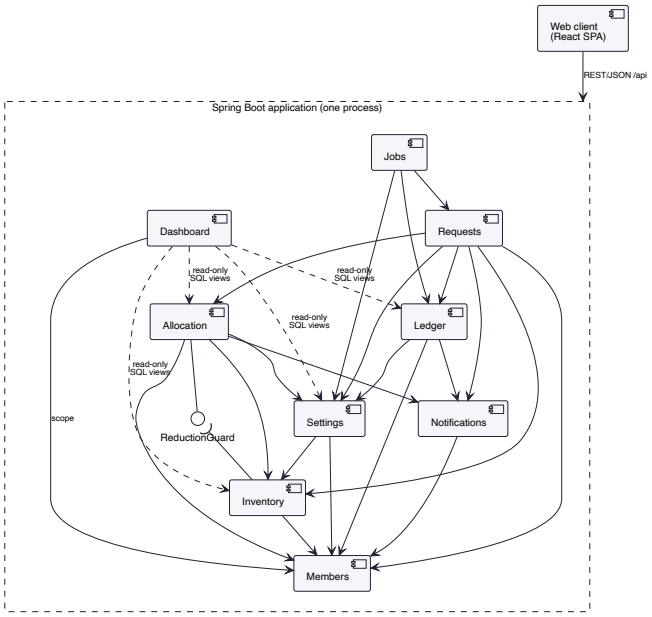


Figure 1: Modules and their dependencies (UML component diagram). A solid arrow is a call to the target's application services. The ball on **Allocation** and the socket on **Inventory** show the port `ReductionGuard`, declared by Inventory and implemented by Allocation. The dashed arrows labeled read-only are **Dashboard**'s SQL access through views, the one exception to calling services.

2.4 Deployment

Figure 2 shows the one deployment of Release 1. A single on-premises server runs one Docker container holding the Spring Boot executable jar. Its embedded Tomcat serves the REST API under `/api` and the compiled single-page application from the same origin, so the browser needs no cross-origin access, and the scheduled jobs run as threads of the same process (**CO-5**). One artifact holds the whole system (**CO-1**). The process talks to PostgreSQL 16 over JDBC. At start-up Flyway applies the SQL migrations and Hibernate validates the schema against the entities (**SI-1**). An optional nginx reverse proxy terminates TLS and redirects plain HTTP to HTTPS (**CI-1**). There is exactly one instance: the SRS sets no high-availability requirement (**LI-1**, **QA-4**), and the two jobs assume no second copy of themselves (Section 8).

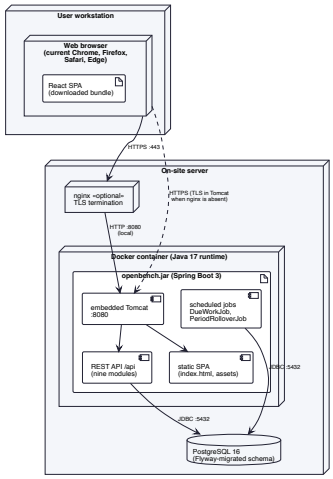


Figure 2: Deployment (UML deployment diagram). One server runs one container holding the Spring Boot jar; embedded Tomcat serves `/api` and the single-page application, and the scheduled jobs run in the same process. PostgreSQL 16 holds all data. The nginx node is optional and only terminates TLS.

2.5 How the Quality Attributes Are Met

Each keyed requirement that shapes the architecture is met by a named mechanism, detailed in the section given; the remaining identifiers are traced in Appendix A.

Requirement	Mechanism	Detailed in
QA-1	Exclusive: a partial exclusion constraint on `exclusive_occurrence` over (Resource, time range) for Held and Allocated rows. Metered: lock each touched `resource_type` row in ascending id order, then check every hour against Pool Capacity and Quota, at READ COMMITTED. Loans: an exclusion constraint on `loan`.	5, 6, ADR-0006
QA-2, **CO-4**	One `@Transactional` method of `RequestLifecycleService` commits the Holds and posts the Charges and Earnings, by direct calls.	4.7, 5
QA-3	A `request_event` row for every state change, in the same transaction; `setting_change`; `transaction` is append-only, with no UPDATE or DELETE grant.	7, 6
QA-6, **CO-7**	A Resource Type is a row; a new Allocation Class is one new `Allocator` bean found by `AllocatorRegistry`, with the optional `Waitlistable`, `Bumpable`, and `ApprovalDriven`.	4.5, ADR-0007
PE-4, **CO-5**	`DueWorkJob` every 60 s against a five-minute bound; `PeriodRolloverJob` at 00:05 and at start-up against a fifteen-minute bound; Spring `@Scheduled` methods inside the server process.	4.9, 5
SE-1	Roles as granted authorities, checked by `@PreAuthorize` on application services: one path for the client and the API.	7
SE-4	The caller's scope (own record, Team, owned Resources, or the whole company) is a SQL condition of every query.	7

2.6 Decisions and Alternatives

The decisions D-A to D-C above and the ones the module specifications and the runtime view state were each weighed against alternatives, which the longer edition tabulates; decisions that are hard to reverse have a record in `docs/adr/`, and the first four records date from the SRS.

Architecture decision records (`docs/adr/` in the project repository): **ADR-0001** Holds bounded by the Objection Window (lines are Required or Optional; a Pending Request places uncharged Holds that never outlive the Objection Window); **ADR-0002** a two-part Account (granted credits expire at the end of the Period, earned credits carry over; a Charge draws granted credits first and records the split); **ADR-0003** Priority bought with a premium, not granted by rank (a Priority line may Bump a standard Exclusive line outside the bump cutoff and is never Bumped; the Bumped line gets a full Refund); **ADR-0004** Matched lines cost nothing (lending earns a flat Day Rate per person-day, posted when the Loan forms); **ADR-0005** one process, nine modules, direct calls (this section's dependency rule and layers); **ADR-0006** allocation correctness enforced in PostgreSQL, and tests run on PostgreSQL; **ADR-0007** one Allocator interface plus optional capability interfaces found through the registry.

3 Problem Decomposition

The problem was broken down in two passes: the first turned the twelve features of the SRS, plus the concerns that cut across them, into responsibilities small enough for one module to own; the second took the four parts that resist a clean split (three allocation policies, a Request whose parts resolve at different speeds, credits, and timed work) and decided how each is divided.

3.1 Responsibilities

Every responsibility has exactly one owning module, the one that holds its data and enforces its rules; where two modules share a responsibility, the first named owns the decision and the second carries it out, and the collaborators a module calls are the required interfaces of Section 4. Every one of the 41 Release 1 functional requirements appears in at least one row, and Appendix A gives the reverse mapping.

Id	Responsibility	SRS identifiers	Owner
Inventory and profiles			
R1	Define, change, and retire Resource Types as data, with one Allocation Class and typed attributes; protect the built-in type Person	**Inventory.Type.Define**, **QA-6**	**Inventory**
R2	Keep Resources, their units, and Pool Capacity; refuse a retirement or reduction below future usage, naming the earliest hour	**Inventory.Resource.Add**	**Inventory**
R3	Show who holds a Resource in each part of a window; filter Resources by type, attribute, and free for a window	**Inventory.View.Holder**, **UI-3**	**Allocation**
R4	Keep Members, Teams with exactly one Manager, roles, the skill list, each Member's skills and availability; answer who is available with which skills	**Admin.Member.Create**, **Profile.Skills**, **Profile.Availability**, **SE-3**	**Members**
Suggestions, drafting, and submission			
R5	Create Suggestions; accept one into a new or existing line, or decline it with a reason	**Suggest.Create**, **Suggest.Accept.Existing**	**Requests**
R6	Edit Drafts: add, edit, split, and remove lines; mark Required or Optional and Priority; refuse past windows and retired types	**Draft.Create**, **Draft.Line.Add**, **Draft.Line.Required**, **UI-1**	**Requests**
R7	Price a line (Quote) and a Release (Refund fraction) with the pure class `Pricing`, from the line, the settings, and the lead time	**Ledger.Price.Formula**, **Draft.Quote.Line**, **Metered.Line.Form**, **Release.Refund**	**Ledger**
R8	Check affordability and submit: refuse a shortfall in Required lines, or any priced Required line while the Balance is negative; lock every Quote; record the submission time	**Draft.Quote.Line**, **Draft.Submit.Lock**, **Release.Loan.Prorate**	**Requests**
Allocation per class			
R9	Decide Matched eligibility (skills, availability, other Team, active, no overlapping Loan) and build the Shortlist least-lent first; form Loans, one per Person at a time	**Matched.Shortlist.Build**, **Profile.Availability**, **Matched.Loan.Form**, **PE-3**, **UI-2**	**Allocation**
R10	Route one approval per selected Candidate to the lending Manager; record each decision; on refusal release Holds and ask for a replacement, or refuse or drop the line	**Matched.Approve.Route**, **Matched.Refuse.Replace**, **UI-2**	**Allocation**, **Requests**
R11	Detect Exclusive conflicts against Held and Allocated occurrences and pick a free instance; never double-allocate	**Exclusive.Conflict**, **QA-1**	**Allocation**
R12	Check a Metered line hour by hour against Pool Capacity and the Team's Quota; never oversell	**Metered.Line.Form**, **Metered.Pool.Check**, **QA-1**	**Allocation**
R13	Keep the Waitlist: join, order (Priority, then submission time, Bumped at the head), positions, and expiry at window start	**Exclusive.Waitlist.Join**, **Metered.Pool.Check**	**Allocation**
R14	Plan and apply Bumps for Priority lines outside the bump cutoff; refund the Bumped line in full and put it at the head of the Waitlist	**Exclusive.Priority.Bump**, **Exclusive.Bump.Effect**	**Allocation**, **Requests**
Request lifecycle			
R15	Run the Request state machine; evaluate every line on submission and choose Allocated, Pending, or Waiting	**Lifecycle.States**, **Lifecycle.Evaluate**	**Requests**
R16	Place Holds with held credits when a Request enters Pending; release them when it leaves Pending; place none after the window lapses	**Lifecycle.Hold.Place**	**Requests**
R17	Allocate a Request in one transaction: commit Holds, post Charges and Earnings, form Loans, drop Optional lines with a reason	**Lifecycle.Allocate**, **Matched.Loan.Form**, **QA-2**, **CO-4**	**Requests**
R18	Start the Objection Window on first entry to Pending; on lapse release Holds, move to Waiting, and escalate to the Capacity Planner	**Lifecycle.Window.Start**, **Matched.Escalate**, **Planner.Settings**	**Requests**
R19	Re-evaluate Waiting Requests when capacity frees or an approval arrives; allocate one only when every Required line fits at once	**Lifecycle.Waiting.Reevaluate**, **Exclusive.Waitlist.Join**	**Requests**
Release and Ledger			

continues on next page

Responsibilities (continued)

Id	Responsibility	SRS identifiers	Owner
R20	Release a line in whole or in part and end a Loan early; post the Refund or the Earning correction; offer the freed capacity	**Release.Line**, **Release.Refund**, **Release.Loan.Prorate**	**Requests**
R21	Keep Accounts (granted, earned, held) and append-only Transactions with the spend-order split; post Earnings; produce Statements	**Ledger.Account**, **Ledger.Transaction.Kinds**, **Ledger.Earning.Post**, **QA-3**	**Ledger**
R22	Post planned and mid-Period Grants; roll the Period over, expiring granted credits and keeping earned ones	**Ledger.Grant.Period**, **PE-4**	**Ledger**
Reporting and settings			
R23	Show utilization, idle capacity, waiting lines, credits, and person-days, scoped to the viewer's roles	**Dashboard.Utilization**, **Dashboard.Credits**, **UI-3**	**Dashboard**
R24	Keep every Capacity Planner setting as data, applied to future Quotes and events, with a change log	**Planner.Settings**, **Admin.Member.Create**	**Settings**
Crosscutting			
R25	Authenticate Members by username and password; enforce roles on every action, from the client or the API alike	**Admin.Member.Create**, **SE-1**, **SE-2**, **CO-6**, **CO-3**	`security`, **Members**
R26	Scope every read to the caller's own record, Team, owned Resources, or role	**SE-4**, **Profile.Availability**, **Ledger.Account**, **Dashboard.Credits**	every module
R27	Record actor, change, and time for every Request and line state change and every setting change	**Lifecycle.States**, **Exclusive.Bump.Effect**, **QA-3**, **QA-7**	**Requests**, **Settings**
R28	Run timed work: lapse Objection Windows, expire waiting lines, roll Periods over, from one injected `Clock`	**Lifecycle.Window.Start**, **Ledger.Grant.Period**, **PE-4**, **CO-5**	**Jobs**
R29	Notify Members in-app of approval requests, outcomes, Bumps, Loans, escalations, and declined Suggestions	**CI-2**, **Lifecycle.Allocate**, **Matched.Loan.Form**, **Suggest.Accept.Existing**	**Notifications**
R30	Present the role-based screens over the REST API, with Quotes fetched from the server	**UI-1**, **UI-2**, **UI-3**, **QA-5**, **QA-4**	Web client
Stretch, designed as extension points (Section 8)			
R31	Service tiers with a premium on Metered lines, and Bumps of lower-tier units by `Bumpable` on `MeteredAllocator`	**Metered.Tier.Define**, **Metered.Tier.Bump**	**Allocation**
R32	Record units actually used per Metered allocation and compare requested with used	**Metered.Usage.Record**, **Metered.Usage.Compare**, **Dashboard.Usage**	**Allocation**, **Dashboard**
R33	Add a demand surcharge multiplier inside `Pricing.quote` that rises as the requested window fills	**Ledger.Surcharge**	**Ledger**
R34	Deliver notifications by email as well as in-app, through a second `Delivery`, `EmailDelivery`	**CI-2** (Stretch part)	**Notifications**

3.2 Subproblems and How Each Was Broken Down

Three allocation policies behind one interface. Every class must evaluate a line, hold it, commit it, release it, and cancel it, so those five operations form `Allocator`. The rest became three optional capabilities (`Waitlistable`, `Bumpable`, `ApprovalDriven`) that **Requests** asks the `AllocatorRegistry` for and skips when a class lacks them (ADR-0007). *Rejected:* three unrelated services that **Requests** picks by class, which would make every new class edit the orchestrator, against **CO-7** and **QA-6**. **A Request whose parts resolve at different speeds.** One orchestrator, `RequestLifecycleService` in **Requests**, owns the state machine and is the only code that moves a Request between states; Holds bridge the gap (ADR-0001), blocking capacity without a Charge and bounded by the Objection Window; and submission, an approval decision, a lapse, and a Release are each one `@Transactional` method that calls the allocators and **Ledger** directly, so a Request is never seen half allocated. *Rejected:* a saga of modules reacting to each other's events, with a compensating step for each failure, which exposes half-allocated Requests. **Credits as pure pricing plus append-only posting.** `Pricing` is a pure class in **Ledger** that reads no database and is used both while drafting and at submission, and the Quote it returns is locked on the Request Line, so the Charge always equals the Quote shown; `LedgerService` updates the Team's `account` row under its lock and appends a Transaction row for every Grant, Charge, Earning, or Refund (ADR-0002). *Rejected:* computing the Charge at commit time from the settings then in force, which lets a Charge differ from the Quote the Manager accepted. **Timed work as polling in one process.** Each timed event was reduced to a row whose time has come; `DueWorkJob` polls every 60 seconds and `PeriodRolloverJob` runs at 00:05 on the boundary day and again at start-up, and each due item is handled in its own transaction by an application-service method of the owning module. *Rejected:* scheduling one timer per due item, which loses timers on restart unless a job store such as Quartz is added.

4 Module Specifications

Each module is specified with the same six fields: purpose, provided interface, required interfaces, data owned, invariants, and failure behavior. Signatures are Java; every application-service method runs in one transaction unless marked read-only. The modules are presented bottom-up, in the order of the layers of Section 2. Figures 3, 4, and 5 give the class diagrams of the three core modules. The longer edition adds, for every module, the responsibilities with their requirement identifiers, an operation table of preconditions and errors, an information-hiding statement, and worked examples.

4.1 Members (`members`)

Purpose. Members records who the people are: Members, Teams, the assignable roles, the skill list, and each Member's availability. It is the identity that `security` authenticates against, and it answers the two questions matching asks of a profile: is this Member available for every day of a window, and which active Members outside a Team hold a set of skills and are available.

Provided interface.

```
interface MemberService {
  Member createMember(NewMember cmd);                        // ADMIN; team required; initial password hashed
  void deactivate(MemberId id);                              // ADMIN; cannot log in, no Shortlist
  void moveToTeam(MemberId id, TeamId team);                 // ADMIN
  void grantRole(MemberId id, Role role);                    // ADMIN; MANAGER: the Team's previous Manager loses it
     (exactly one per Team)
  void revokeRole(MemberId id, Role role);                   // ADMIN; refused if it would leave a Team with no
     Manager
  Team createTeam(String name); void renameTeam(TeamId, String); void retireTeam(TeamId);  // ADMIN; retire
     refused while Members remain
  void updateSkills(MemberId id, Set<SkillId> skills);       // self
  void updateAvailability(MemberId id, List<DateRange> ranges);  // self or own Manager
  MemberProfile profile(MemberId id);                        // (read-only) self, own Manager, PLANNER, ADMIN
  MemberSummary summary(MemberId id);                        // (read-only, unscoped) name, Team, active, roles; for
     other modules' rules
  List<MemberSummary> roster(TeamId team);                   // (read-only) that Team's Manager, PLANNER, ADMIN
  boolean isAvailable(MemberId id, DateRange window);        // (read-only) every day inside one range
  Set<MemberId> availableWithSkills(Set<SkillId> skills, DateRange window, TeamId excludeTeam); // (read-only)
     active Members only
  List<Skill> skills(); Skill addSkill(String name); void retireSkill(SkillId id);  // ADMIN for writes
  List<MemberId> withRole(Role role);                        // (read-only) active Members holding the role
  MemberId managerOf(TeamId team);                           // (read-only) the Team's current Manager
  List<Team> teams();                                        // (read-only)
}
```

Required interfaces. None. Members hashes passwords with the bcrypt `PasswordEncoder` bean of the `security` configuration.

Data owned. `team`, `member`, `member_role`, `skill`, `member_skill`, `availability_range`.

Invariants.

- Each Team has exactly one Manager at any time once the role is first granted in it: the Manager is one column, so there cannot be two, and the role moves only to a successor. A Team that has never had a Manager can own no Request, since only its Manager can create one.
- A deactivated Member cannot log in and appears on no Shortlist: the `UserDetailsService` reports them disabled, `availableWithSkills` returns active Members only, and `isAvailable` returns false for them.
- A Member's ranges never overlap or touch, so "available for a window" is the single test "one range contains it"; a Loan never changes them. At least one active Administrator always exists.

Failure behavior. A refusal is a `RuleViolation` with one of the codes `USERNAME_TAKEN`, `TEAM_RETIRED`, `TEAM_NEEDS_MANAGER`, `LAST_ADMINISTRATOR`, `MEMBER_INACTIVE`, `TEAM_NAME_TAKEN`, `TEAM_NOT_EMPTY`, `SKILL_RETIRED`, `SKILL_NAME_TAKEN`, or `RANGE_INVALID`; the transaction rolls back whole, and the code reaches the client in the `problem+json` body (Section 7). Unknown ids raise `NotFoundException`, and so does an object outside the caller's scope (404); acting on a visible object without the right to do so raises `ForbiddenException`.

4.2 Inventory (inventory)

Purpose. Inventory holds Resource Types as data and the Resources that exist, with their type, attribute values, owner, and, for Metered Resources, units. It derives Pool Capacity and refuses a reduction below future usage, which it learns through the port `ReductionGuard` so that it never depends on the layer above.

Provided interface.

```
interface ResourceTypeService {
  ResourceType define(ResourceTypeDefinition def);          // ADMIN; unique name, one Allocation Class, attributes
  ResourceType change(ResourceTypeId id, ResourceTypeDefinition def);  // ADMIN; class immutable once a Resource
     exists
  void retire(ResourceTypeId id);                           // ADMIN; refused for built-in Person; retired types
     accept no new lines
}
interface ResourceService {
  Resource add(NewResource cmd);                            // RESOURCE_OWNER; attributes per type; units for
     Metered
  Resource edit(ResourceId id, ResourceEdit cmd);           // owner of that Resource
  void setUnits(ResourceId id, int units);                  // owner; calls ReductionGuard; refused naming the
     earliest failing hour
  void retire(ResourceId id);                               // owner; same guard with units 0
}
interface InventoryQueries {                                // all read-only
  ResourceType type(ResourceTypeId id); List<ResourceType> types(boolean includeRetired);
  List<Resource> instancesOf(ResourceTypeId id);           // unretired
  int poolCapacity(ResourceTypeId id);                      // sum of units of unretired Resources
  List<Resource> search(ResourceFilter f);                  // by type, attribute value; free-for-window answered
     by Allocation
}
interface ReductionGuard {                                   // port declared here, implemented in allocation
  Optional<Instant> earliestHourExceeding(ResourceTypeId type, Optional<ResourceId> instance, int newUnits,
     Instant from);
}
```

Required interfaces. **Members**: `MemberService.summary`. **Allocation**, through the port only: `ReductionGuard.earliestHourExceeding`.

Data owned. `resource_type`, `resource_type_attribute`, `resource`, `resource_attribute_value`.

Invariants.

- A Resource Type's Allocation Class never changes once a Resource of it exists. The built-in Person type (class Matched) cannot be retired, and its class never changes.
- No Resource Type or Resource is ever deleted, only retired. A Matched type has no Resource rows (its Resources are Members); an Exclusive Resource has units 1.
- Pool Capacity equals the sum of the units of the unretired Resources of the type. It is one `SUM` on every call and never stored, so it cannot drift.
- A reduction or retirement is refused, naming the earliest hour at which allocated plus held units would exceed the new Pool Capacity, or at which the Exclusive instance is in use. A Hold counts exactly as an allocation.
- `instancesOf` never returns a retired Resource, so no allocator can pick one.

Failure behavior. A refusal is a `RuleViolation` with one of the codes `TYPE_NAME_TAKEN`, `ATTRIBUTE_INVALID`, `CLASS_FIXED`, `PERSON_TYPE_FIXED`, `ATTRIBUTE_IN_USE`, `TYPE_RETIRED`, `MATCHED_HAS_NO_RESOURCES`, `RESOURCE_NAME_TAKEN`, `UNITS_INVALID`, `NOT_A_RESOURCE_OWNER`, `NOT_METERED`, `UNITS_BELOW_USAGE`, or `RESOURCE_IN_USE`, and nothing is written; the two guard refusals carry an `earliestHour` property so the screen names the blocking hour. A lock conflict surfaces as `ConcurrencyRetryException` after one retry.

4.3 Settings (settings)

Purpose. Settings holds every Capacity Planner setting as data and logs every change. Other modules read one typed snapshot, `PlannerSettings`, and never see how settings are stored.

Provided interface.

```
interface SettingsService {
  PlannerSettings current();                                  // (read-only) typed snapshot, see below
  void update(SettingChange change);                          // PLANNER; writes setting_change(actor, key, old, new,
     at)
  List<Period> periods();                                     // (read-only) past and planned Periods
}
record PlannerSettings(Period currentPeriod, int objectionWindowWorkingDays, Credits dayRate,
                       BigDecimal priorityPremium, Map<ResourceTypeId, TypePricing> pricing,
                       Map<TeamTypeKey, Integer> quotas, Map<TeamId, Credits> grants) {}
record TypePricing(Credits unitPrice, List<PriceBand> bands, RefundSchedule refundSchedule, Duration bumpCutoff) {}
```

`SettingChange` is a sealed interface with one record per kind of setting, so `update` is one entry point with typed arguments.

```
sealed interface SettingChange {      // every record below implements SettingChange
  record PeriodLength(int months) {}               record DayRate(Credits rate) {}
  record PriorityPremium(BigDecimal multiple) {}   record ObjectionWindow(int workingDays) {}
  record GrantPlan(TeamId team, PeriodId period, Credits amount) {}
  record UnitPrice(ResourceTypeId type, Credits price) {}
  record PriceBands(ResourceTypeId type, List<PriceBand> bands) {}
  record Refunds(ResourceTypeId type, RefundSchedule schedule) {}
  record BumpCutoff(ResourceTypeId type, Duration cutoff) {}
  record Quota(TeamId team, ResourceTypeId type, Optional<Integer> maxUnits) {}   // empty: no Quota
}
record Period(PeriodId id, LocalDate startsOn, LocalDate endsOn, Optional<PeriodId> previous) {}
record PriceBand(Set<DayOfWeek> weekdays, LocalTime start, LocalTime end, BigDecimal multiplier) {}  // end
     exclusive
record RefundSchedule(List<RefundStep> steps) {}      record RefundStep(Duration minLead, BigDecimal fraction) {}
```

Required interfaces. **Inventory**: `InventoryQueries.type`. **Members**: no call.

Data owned. `period`, `grant_plan`, `type_pricing`, `price_band`, `refund_step`, `quota`, `planner_setting`, `setting_change`.

Invariants.

- Every setting change is logged: `update` writes exactly one `setting_change` row (actor, key, old value, new value, timestamp) in the same transaction as the change, so neither exists without the other.
- A changed price affects only Quotes not yet locked: locked Quotes live on the Request Line, owned by **Requests**, and Settings never reads or writes them.
- The Price Bands of a type never overlap; an hour no band covers has multiplier 1.0. No refund step reaches 100%.
- Periods are contiguous and never overlap, so exactly one contains any date; a started Period never changes its dates.

Failure behavior. A refused change is a `RuleViolation` with one of the codes `VALUE_OUT_OF_RANGE`, `PERIOD_STARTED`, `CLASS_MISMATCH`, `BAND_INVALID`, `BANDS_OVERLAP`, or `REFUND_STEP_TOO_HIGH`; the change and its log row roll back together. `update` locks the changed row before reading the old value, so concurrent edits of one setting are serialized and each logged old value is the one replaced.

4.4 Notifications (notifications)

Purpose. Notifications tells a Member, inside the application, that something concerning them has happened. Callers decide when a notice is due and what it says, and call `Notifier.notify` inside their own transaction; Notifications decides how the notice is stored and delivered.

Provided interface.

```
interface Notifier { void notify(MemberId to, NotificationKind kind, String text, String link); }
interface NotificationQueries { List<Notification> unread(MemberId me); void markRead(NotificationId id); }
interface Delivery { void deliver(Notification n); }         // InAppDelivery (row only); EmailDelivery is Stretch
```

Required interfaces. **Members**, for recipient identity only: `member_id` references `member(id)` and no operation is called.

Data owned. `notification`.

Invariants.

- Notifications are rows written in the caller's transaction: a notice exists exactly when the action that raised it committed, and nothing is sent after commit in Release 1.
- A notice is visible only to its recipient; `read_at` is set at most once, and no notice is deleted.

Failure behavior. No `RuleViolation` codes. If the insert fails, the caller's action rolls back with it. `markRead` on another Member's notice raises `NotFoundException`, which does not reveal that the notice exists.

4.5 Allocation (`allocation`)

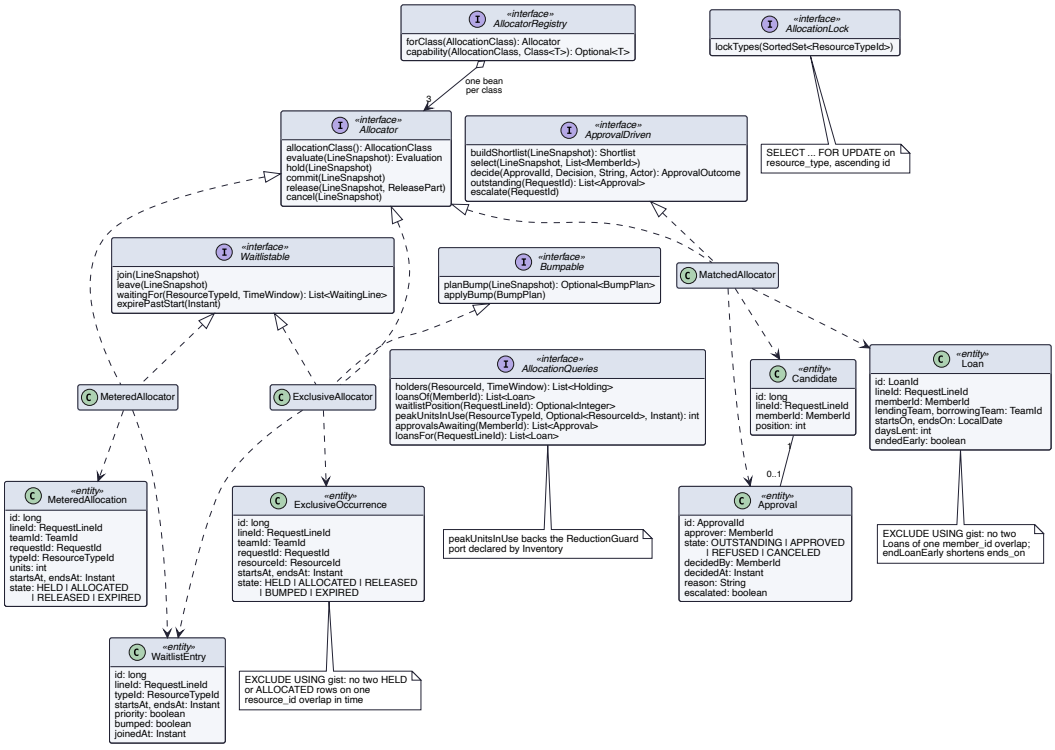


Figure 3: Classes of **Allocation**: the `Allocator` interface with its five operations, the three capability interfaces, the registry, the three allocators and the capabilities each realizes, the shared lock and query services, and the six entities the module owns. Dashed arrows are dependencies; hollow-triangle dashed arrows are realizations.

Purpose. Allocation decides, one Request Line at a time, whether the capacity it asks for can be had now, and records the answer as rows; a Hold is simply the state `HELD` of such a row. Everything that differs between Allocation Classes lives here, behind one interface that **Requests** drives without branching on the class.

Provided interface.

```
interface Allocator {
  AllocationClass allocationClass();
  Evaluation evaluate(LineSnapshot line);                    // SATISFIABLE | NEEDS_APPROVAL | UNSATISFIABLE(reason)
  void hold(LineSnapshot line);                              // Exclusive, Metered: rows in state HELD; Matched:
     no-op
  void commit(LineSnapshot line);                            // HELD -> ALLOCATED; Matched: form Loans
  void release(LineSnapshot line, ReleasePart part);         // whole or part; Matched: end Loan early
  void cancel(LineSnapshot line);                            // drop Holds, approvals, waitlist entries
}
interface Waitlistable { void join(LineSnapshot line); void leave(LineSnapshot line);
  List<WaitingLine> waitingFor(ResourceTypeId type, TimeWindow freed); void expirePastStart(Instant now); }
```

```
interface Bumpable { Optional<BumpPlan> planBump(LineSnapshot priorityLine); void applyBump(BumpPlan plan); }  // Exclusive only
interface ApprovalDriven {
  Shortlist buildShortlist(LineSnapshot line);              // eligible = skills, available, no overlapping Loan, other Team, active; order by person-days lent this Period, then name
  void select(LineSnapshot line, List<MemberId> candidates); // exactly the line's quantity; creates one approval per Candidate
  ApprovalOutcome decide(ApprovalId id, Decision d, String reason, Actor actor);  // lending Manager or PLANNER (escalated)
  List<Approval> outstanding(RequestId request); void escalate(RequestId request);
}
interface AllocatorRegistry { Allocator forClass(AllocationClass c); <T> Optional<T> capability(AllocationClass c, Class<T> cap); }
interface AllocationLock { void lockTypes(SortedSet<ResourceTypeId> types); }   // SELECT ... FOR UPDATE on resource_type, ascending id
interface AllocationQueries {                               // read-only
  List<Holding> holders(ResourceId resource, TimeWindow window);   // free / HELD / ALLOCATED segments with Team and Request
  List<Loan> loansOf(MemberId member); Optional<Integer> waitlistPosition(RequestLineId line);
  List<Loan> loansFor(RequestLineId line);                  // the Loans a Matched line's commit formed (for postEarning)
  int peakUnitsInUse(ResourceTypeId type, Optional<ResourceId> instance, Instant from);  // backs ReductionGuard
  List<Approval> approvalsAwaiting(MemberId approver);      // outstanding approvals addressed to this Manager; every escalated one for a PLANNER
}
```

`LineSnapshot` is an immutable copy of a line built by **Requests** (ids, type and class, quantity or units, window and recurrence, named Resource or Person, skills, Required and Priority marks, submission time, a `bumped` flag); `ReleasePart` is the whole line, some occurrences, an earlier end, fewer units, or one Loan with a new last day.

Registry and extension. `AllocatorRegistry` receives `List<Allocator>`, which Spring fills with every `Allocator` bean, and indexes it by `allocationClass()`. `capability(c, cap)` returns the allocator cast to `cap` if it implements it, so **Requests** never names a class: the Priority mark is offered for a `Bumpable` class, waiting for a `Waitlistable` one, and a Quote whenever the type has pricing settings. A fourth Allocation Class is one new `@Component` implementing `Allocator` and its capabilities, plus one `AllocationClass` constant; **Requests**, **Ledger**, and the existing allocators do not change (**CO-7**, **QA-6**).

Required interfaces. **Inventory**: `InventoryQueries.type`, `instancesOf`, `poolCapacity`. **Members**: `MemberService.availableWithSkills`, `isAvailable`, `summary`, `managerOf`, `withRole(PLANNER)`. **Settings**: `SettingsService.current`. **Notifications**: `Notifier.notify`.

Data owned. `exclusive_occurrence`, `metered_allocation`, `waitlist_entry`, `candidate`, `approval`, `loan`.

Invariants.

- No two live (`HELD` or `ALLOCATED`) occurrences on one Exclusive instance overlap; in no hour do a Metered type's live units exceed its Pool Capacity, or a Team's its Quota (**QA-1**).
- Every write that consumes capacity, and every capacity reduction, comes after the lock on that Resource Type in the same transaction.
- A line's live rows are all or nothing: every occurrence on the same instances, or the full units for the full window.
- A Priority line is never Bumped, and no line is Bumped inside the bump cutoff.
- A Person is in at most one Loan on any day. A Candidate is active and outside the requesting Team, and until escalation only the Manager of the Candidate's Team decides.

Failure behavior. A misfit is a result of `evaluate`, never an exception, with reason `NO_FREE_INSTANCE`, `POOL_FULL`, `QUOTA_EXCEEDED`, `SHORTLIST_TOO_SHORT`, or `NOT_A_CANDIDATE`; the codes `NOT_ALLOCATED`, `INSIDE_BUMP_CUTOFF`, `LINE_NOT_ATOMIC`, `OWN_TEAM`, `ALREADY_DECIDED`, and `WINDOW_LAPSED` report bad input or a precondition the caller should have established. A lock wait past `lock_timeout` (5 s) or a deadlock is retried once, then reported as `ConcurrencyRetryException` (503).

Shared parts

`AllocationLock` runs one `SELECT ... FOR UPDATE` on `resource_type` per id, ascending (`@Lock(PESSIMISTIC_WRITE)`), at the default READ COMMITTED isolation; a transition calls it once, first, for the types of its own lines, and keeps the locks to commit, so allocation on a type is serial. `AllocationQueries` reads live rows: `holders` cuts a window into free, held, and allocated segments; `peakUnitsInUse` is the largest

hourly sum of live units, Holds counted, from an instant on. The `ReductionGuard` implementation locks the type, computes the capacity after the change, and, if `peakUnitsInUse` exceeds it, names the earliest failing hour with the hourly query.

ExclusiveAllocator

The window and weekly recurrence expand into occurrences; an instance is free when no live row on it overlaps any of them. `hold` inserts one `HELD` row per occurrence and instance, `commit` turns them `ALLOCATED`, and `release` marks given-back occurrences (or a shortened tail) `RELEASED`, so history stays. For a Priority line with no free instance, `planBump` collects per instance the lines whose live rows overlap its occurrences. An instance can be cleared when all are standard and none has an occurrence starting before now plus the bump cutoff; the plan takes the instances needing the fewest Bumps, ties toward the latest-submitted lines. `applyBump` marks the Bumped lines' live rows `BUMPED` with a bulk JPQL update and only then inserts the Priority occurrences as `HELD`: Hibernate runs inserts before updates, so without the flush the insert would meet the NOT DEFERRABLE constraint while the old rows are still live. `waitingFor` returns the type's entries overlapping the freed window ordered by `bumped`, then Priority, then `joined_at`, then id; `joined_at` holds the Request's submission time, not the moment of joining.

MeteredAllocator

A line asks for units held concurrently over its whole window, one `metered_allocation` row. Under the type lock, `evaluate` runs the hourly query of Section 6 and `HourlyLoad` finds the earliest hour where `used` plus the line's units exceed `poolCapacity(type)` (`POOL_FULL`) or, when the Team has a Quota on the type, where `team_used` plus the units exceed it (`QUOTA_EXCEEDED`). The line fits with full units in every hour or not at all. Holds, `release` (fewer units or an earlier end), and the waitlist follow the Exclusive rules; there is no Bump.

MatchedAllocator

A Candidate holds every listed skill, is available on every day of the window, has no Loan overlapping any day of it, belongs to another Team, and is active. `buildShortlist` takes `availableWithSkills(skills, window, requestingTeam)` from **Members**, drops Members with an overlapping Loan, and orders the rest by person-days lent in the current Period (weekdays of formed Loans inside it), then name: two indexed queries and a sort of at most 500 Members (**PE-3**). A Person can be approved on two Requests with overlapping windows before either Loan forms. **Requests** therefore re-evaluates the line under the type lock at the start of `allocate`, before any commit, where `evaluate` reports an Approved Candidate who is no longer available every day, or now has an overlapping Loan, as `UNSATISFIABLE(NOT_A_CANDIDATE)`: a Required line is then unsatisfied and the Request goes to Waiting with a replacement asked; an Optional line is Dropped. `commit` repeats the check immediately before forming each Loan only as a guard: a failure there means a path skipped the re-evaluation, so it throws `RuleViolation("ALLOCATION_CONFLICT")` and the transition rolls back.

4.6 Ledger (`ledger`)

Purpose. Ledger keeps every Team's Account and the append-only record of Transactions, prices lines, and computes Refunds. It knows credits only: it holds no capacity information and never decides whether a line can be allocated.

Provided interface.

```
final class Pricing {                                          // pure, no Spring, no database
  static Quote quote(LineSpec line, TypePricing pricing, BigDecimal priorityPremium);  // sum over hours of
     quantity x unitPrice x band, x premium if Priority
  static Credits refund(Credits charge, RefundSchedule schedule, Duration leadTime);  // step function, never 100%
}
interface LedgerService {
  AccountView account(TeamId team);                            // (read-only) granted, earned, held, balance, available
  void requireAffordable(TeamId team, Credits requiredSum); // no lock, early refusal; throws
     RuleViolation("INSUFFICIENT_BALANCE")
  void holdCredits(TeamId team, RequestLineId line, Credits quote);     // held += quote (locks account row,
     repeats the check)
  void releaseCredits(TeamId team, RequestLineId line);                 // held -= that line's quote
  void commitCharges(RequestId request, List<ChargeSpec> charges);      // for each line: held -> Charge, granted
     first then earned, split recorded
  void postEarning(TeamId lendingTeam, LoanId loan, Credits amount);    // earned += amount
  void refund(TeamId team, RequestLineId line, Credits amount);         // returns in the Charge's proportions
  void correctEarning(TeamId lendingTeam, LoanId loan, Credits delta);  // only Transaction that may make Balance
     negative
  void postGrant(TeamId team, PeriodId period, Credits amount, String reason, Actor actor);  // PLANNER or rollover
```

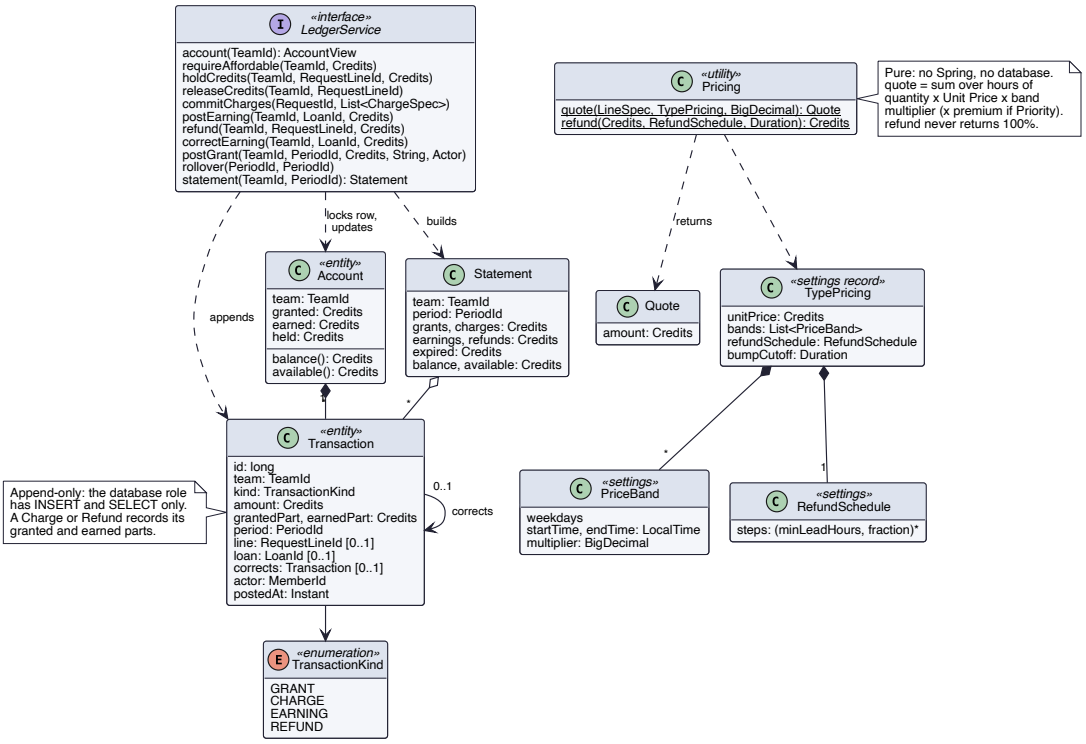


Figure 4: Classes of **Ledger**: `LedgerService`, the pure `Pricing` functions, the `Account` with its three numbers, the append-only `Transaction` with its four kinds, and the value types `Quote`, `PriceBand`, and `RefundSchedule`.

```
  void rollover(PeriodId ending, PeriodId starting);        // expire granted, record on Statement, post planned
     Grants; idempotent (no-op if already rolled)
  Statement statement(TeamId team, PeriodId period);        // (read-only) own Manager or PLANNER
}
```

With q the quantity, p the Unit Price, $b(h)$ the multiplier of the Price Band containing hour h (1.0 if none), and π the priority premium for a Priority line (1 otherwise):

$$Quote = \pi \cdot \sum_{\text{occurrence } o} \; \sum_{\text{hour } h \in o} \; q \cdot p \cdot b(h).$$

Credits carry two decimals; a Quote is rounded half-up once per line. `refund` takes the step with the largest lead time not above `leadTime` and returns that fraction of `charge`, rounded down; a part of an occurrence already started returns 0.

Required interfaces. **Settings**: `SettingsService.current`. **Members**: a Team's Manager, for Statement scoping. **Notifications**: `Notifier.notify`.

Data owned. `account`, `transaction`, `held_credit`, `period_close`.

Invariants.

- Granted and held credits are never negative; the Balance is never negative except after an Earning correction.
- Held credits equal the Team's `held_credit` rows, which equal the locked Quotes of its Held lines.
- The account row equals the fold of the Team's Transactions and `period_close` rows (a test recomputes it).
- A Charge equals its locked Quote and its two parts sum to it, granted first; a line's Refunds stay within its Charge, in its proportions. Transaction rows never change.
- `rollover` is idempotent: rolling into a Period already rolled into is a no-op (the key of `period_close`), so the start-up catch-up in **Jobs** is safe.

Failure behavior. `INSUFFICIENT_BALANCE` is the refusal a Manager sees, with the shortfall; the other codes are `REFUND_EXCEEDS_CHARGE`, `INVALID_AMOUNT`, and `REASON_REQUIRED` (a mid-Period Grant without a reason). The methods join the caller's transaction, so a failed Charge rolls back the commit of every Hold (**CO-4**).

4.7 Requests (requests)

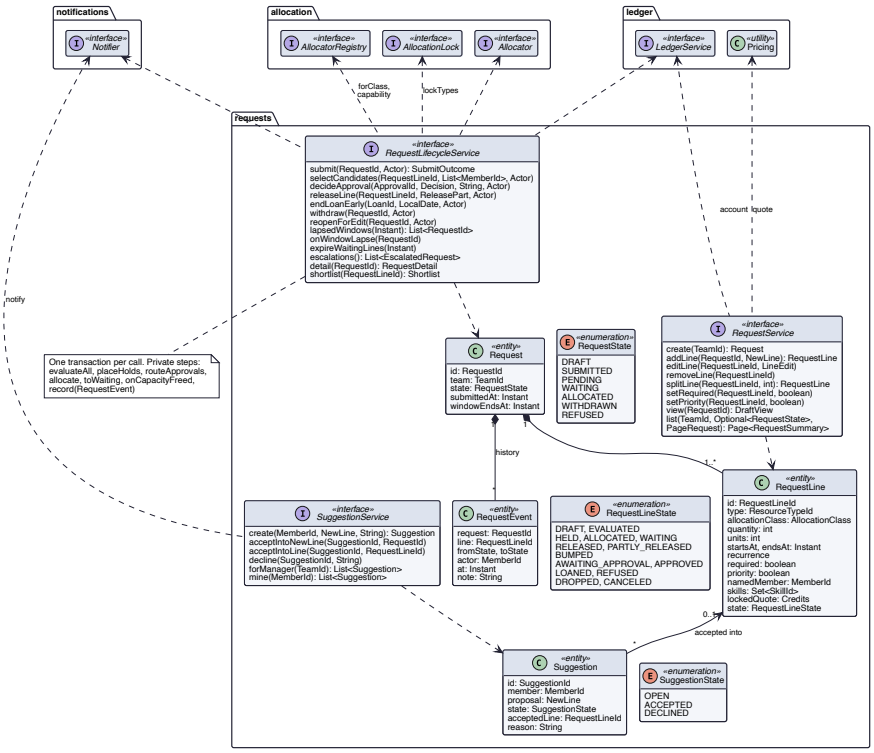


Figure 5: Classes of **Requests**: the Draft-editing and Suggestion services, the orchestrating `RequestLifecycleService`, the entities and state enums, and the module's dependencies on `Allocator`, `AllocatorRegistry`, `AllocationLock`, `LedgerService`, and `Notifier`.

Purpose. Requests owns the Request from Draft to a final state and the Suggestions that feed it. It alone changes a Request's state and alone calls both **Allocation** and **Ledger**, so the order of locks, checks, writes, and notices lives in one class.

Provided interface.

```
interface RequestService {                                    // Draft editing; MANAGER of the owning Team
  Request create(TeamId team);
  RequestLine addLine(RequestId r, NewLine line);            // window in the future, type not retired
  void editLine(RequestLineId l, LineEdit e); void removeLine(RequestLineId l);
  RequestLine splitLine(RequestLineId l, int quantityForNewLine);   // new line same type, marked Optional by
     default
  void setRequired(RequestLineId l, boolean required); void setPriority(RequestLineId l, boolean priority);  //
     Priority: Exclusive only
  DraftView view(RequestId r);                                 // (read-only) lines with current Quotes, sums,
     Available Balance, shortfall
  Page<RequestSummary> list(TeamId team, Optional<RequestState> state, PageRequest page);  // (read-only) scoped
     to the caller
}
interface SuggestionService {
  Suggestion create(MemberId by, NewLine proposal, String note);       // MEMBER, to own Manager
  void acceptIntoNewLine(SuggestionId s, RequestId draft); void acceptIntoLine(SuggestionId s, RequestLineId
     line);  // MANAGER
  void decline(SuggestionId s, String reason);                 // MANAGER; notifies the Member
  List<Suggestion> forManager(TeamId team); List<Suggestion> mine(MemberId me);   // (read-only)
}
interface RequestLifecycleService {                            // one transaction per call; locks types then account
  SubmitOutcome submit(RequestId r, Actor actor);              // affordability, lock Quotes, evaluate, -> Allocated
     | Pending (Required approvals) | Waiting
  void selectCandidates(RequestLineId l, List<MemberId> members, Actor actor);
  void decideApproval(ApprovalId a, Decision d, String reason, Actor actor);   // last Required approval
     allocates, a late Optional one forms its Loan; refusal -> replacement
```

```
  void releaseLine(RequestLineId l, ReleasePart part, Actor actor);          // Refund, free capacity, offer to
    waitlist
  void endLoanEarly(LoanId loan, LocalDate newEnd, Actor actor);             // Earning correction
  void withdraw(RequestId r, Actor actor);                  // from Draft, Pending, Waiting; cancels all lines
  void reopenForEdit(RequestId r, Actor actor);             // Waiting -> Draft
  List<RequestId> lapsedWindows(Instant now);               // (read-only) jobs: Requests, Pending, Waiting, or
    Allocated, whose Objection Window ended at or before now with an approval still Outstanding and not yet
    escalated
  void onWindowLapse(RequestId r);                          // jobs, idempotent: Pending: toWaiting, then
    escalate; Waiting or Allocated: escalate only; skips a Request with no unescalated Outstanding approval
  void expireWaitingLines(Instant now);                     // jobs
  List<EscalatedRequest> escalations();                     // (read-only) PLANNER
  RequestDetail detail(RequestId r);                        // (read-only) states, lines, Holds, events timeline
  Shortlist shortlist(RequestLineId l);                     // (read-only) pass-through to
    ApprovalDriven.buildShortlist for the requesting Manager
}
```

`RequestLifecycleService` is the orchestrator: one `@Transactional` method per transition, retried once after a deadlock or lock timeout, always in this order: (1) lock the Resource Types of its own lines with one ascending `AllocationLock.lockTypes`; (2) lock the Team's account row through the first locking Ledger call (`holdCredits` or `commitCharges`); (3) read; (4) write; (5) `record`; (6) `Notifier`, whose rows share the transaction, so a rolled-back transition tells nobody.

Private steps.

`evaluateAll` `evaluate` every line; a Priority misfit asks `planBump`, and a plan counts as a fit.
`placeHolds` Only while the window runs: Required, then affordable Optional lines; `applyBump` where planned, `hold`, `holdCredits`.
`routeApprovals` `ApprovalDriven.select` for named Persons at submission and for the Manager's picks.
`allocate` Always in this order. (a) Re-run `evaluateAll` under the locks; an Approved Candidate who is no longer eligible makes the line `UNSATISFIABLE(NOT_A_CANDIDATE)`, and the system refuses that approval. If a Required line fails, `toWaiting` runs instead, before any commit, and a replacement is asked. If the Balance no longer covers the Required Charges, `toWaiting` runs instead with reason `INSUFFICIENT_BALANCE`. (b) Drop unfit or unaffordable Optional lines, and those refused with no Candidate left, with the reason. (c) Apply Bump plans, then `Allocator.commit` each Exclusive and Metered line (HELD to ALLOCATED). (d) `LedgerService.commitCharges`. (e) `MatchedAllocator.commit` each approved Matched line, forming the Loans. (f) `postEarning` per Loan from `loansFor(line)`. (g) Set the Request Allocated, `record`, and notify the Manager and each Person through `Notifier`. An Optional Matched line still awaiting approval stays open; its late last approval runs (e) and (f) for it alone.
`toWaiting` `cancel` and `releaseCredits` for Held lines; `join` for unsatisfied Required lines; approvals stay; `onCapacityFreed`.
`onCapacityFreed` For each line from `waitingFor(type, window)`, in order: allocate its Request if every Required line fits at once and is approved; if only approvals are missing and the window runs, back to Pending with `placeHolds`; else next.
`record` Insert a `request_event`; a null actor is the system (jobs).

Window, escalation, Release. `window_ends_at` is set at submission when any line needs approval, to that instant plus `objectionWindowWorkingDays` Monday-to-Friday days, and is never reset; `placeHolds` does nothing after it. `escalate` sets `escalated` on every Outstanding approval and hands it to every Capacity Planner, who decides from `escalations()`; from then on only a moment when every Required line fits at once allocates the Request.

Required interfaces. **Allocation**: all of its interfaces. **Ledger**: `Pricing`; `LedgerService` except `postGrant`, `rollover`, `statement`. **Inventory**: `InventoryQueries.type`. **Members**: `summary`. **Settings**: `current`. **Notifications**: `Notifier.notify`.

Data owned. `request`, `request_line`, `suggestion`, `request_event`.

Invariants.

- A Request is in exactly one state; every Request or line state change writes a `request_event` in the same transaction.
- `locked_quote` and `submitted_at` are set at submission and cleared only by `reopenForEdit`; `window_ends_at` never changes once set, and Held lines exist only in a Pending Request before it.

- Lapse is judged by the injected `Clock`, not by the job: `decideApproval` compares `window_ends_at` with now, so an approval arriving after the window ended is treated as lapsed (`WINDOW_LAPSED` for the lending Manager, no Hold kept) even before `onWindowLapse` has run.
- An Allocated Request had every Required line Allocated or Loaned when it entered the state. Only Optional lines are Dropped; only Required lines wait.

Failure behavior. Any exception rolls back the whole transition, notifications included. A `RuleViolation` becomes `problem+json` with its `code` (409 state conflict, 422 invalid input), one of `WRONG_STATE`, `WINDOW_NOT_FUTURE`, `WINDOW_INVALID`, `TYPE_RETIRED`, `OWN_TEAM`, `PRIORITY_NOT_OFFERED`, `QUANTITY_INVALID`, `NAMED_RESOURCE_NOT_OFFERED`, `LINE_NOT_ATOMIC`, `SUGGESTION_CLOSED`, `TYPE_MISMATCH`, `EMPTY_REQUEST`, `INSUFFICIENT_BALANCE`, `NOT_ALLOCATED`, `LOAN_END_INVALID`, or a code of **Allocation**; a deadlock or lock timeout, after one retry, `ConcurrencyRetryException` (503).

4.8 Dashboard (`dashboard`)

Purpose. Dashboard answers the showback questions: how much of each Resource Type was used and how much stood idle, how many lines wait, and what each Team was granted, charged, and earned. It is the one read-only reporting module the dependency rule allows (Section 2): it reads other modules' tables through SQL views, writes nothing, and limits every answer to the caller's scope.

Provided interface.

```
// dashboard (read-only)
interface DashboardQueries {
  Utilization utilization(ResourceTypeId type, PeriodId period);   // allocated hours / available hours; idle =
     available - allocated
  Idle idleNextDays(ResourceTypeId type, int days); int waitingCount(ResourceTypeId type);
  TeamCredits credits(TeamId team, PeriodId period);        // Grants, Charges, Earnings, Refunds, expired,
     Balance, Available
  PersonDays personDays(TeamId team, PeriodId period);      // lent and borrowed
}
```

Required interfaces. Read-only SQL through the views below. **Members**: `MemberService.profile`.

Data owned. No tables; the views `v_type_hours`, `v_team_credits`, and `v_waiting`.

Invariants.

- Dashboard writes nothing. Every method is `@Transactional(readOnly = true)`, which the PostgreSQL driver runs as a `READ ONLY` transaction, so an accidental write fails in the database; the ArchUnit test also forbids saving repositories here.
- Figures include everything committed before the query began: plain views are read at request time and nothing is cached. A Hold is not utilization.

Failure behavior. An unknown type or Period raises `NotFoundException`, and so does an object outside the caller's scope (404). The one `RuleViolation`, `CLASS_MISMATCH`, refuses utilization of a Matched type.

4.9 Jobs (`jobs`)

Purpose. Jobs runs the timed work inside the server process: lapsing Objection Windows, removing waiting lines whose window has started, and rolling a Period over. It decides when work is due and calls the module that owns it; it never decides what the work does.

Provided interface. Jobs has no callers; its interface is its schedule.

```
class DueWorkJob { @Scheduled(fixedDelay = 60_000) void run(); }   // lapsedWindows(now) -> onWindowLapse;
     expireWaitingLines
class PeriodRolloverJob { @Scheduled(cron = "0 5 0 * * *") void run(); @EventListener(ApplicationReadyEvent) void
     catchUp(); }
```

Both jobs run under a system security context, so `@PreAuthorize` admits them and every audit row they cause names the system as actor.

Required interfaces. **Requests**, through `RequestLifecycleService`: `lapsedWindows`, `onWindowLapse`, and `expireWaitingLines`. **Ledger**: `LedgerService.rollover`. **Settings**: `SettingsService.current`.

Data owned. None.

Invariants.

- Jobs are idempotent: running one twice for the same due item has the effect of running it once. Each call re-checks state in its own transaction, so a Request with no unescalated Outstanding approval is skipped, and `rollover` ignores a boundary it has already recorded.
- Each Request is its own transaction, so one failure never blocks the others. No job acts before the due time read from the `Clock`.
- One instance runs the jobs; a second would need a job lock table (Section 8).

Failure behavior. Jobs defines no `RuleViolation` codes. When the call for one Request throws, a concurrency failure included, the error is logged with the Request id and the run moves on; the Request still has a lapsed window and an unescalated Outstanding approval, so the next run retries it.

4.10 Web client (`web`)

Purpose. The Web client is the single-page browser application through which every role works; its only interface to the system is the REST API, and it holds no business rule.

Provided interface. The client's interface is its routes. Each screen calls the API groups of the modules named (endpoints in Section 7); the roles are those the server enforces, which the client uses only to choose menu items.

Screen	Route	API groups	Roles
Requests	`/requests`	**Requests**	Manager; Planner reads
Draft Request	`/requests/:id` (Draft)	**Requests** (Draft view, line edits, split, submit, withdraw, Suggestions), **Inventory** (types)	Manager
Request detail	`/requests/:id`	**Requests** (detail, Release, end Loan early), **Allocation** (Waitlist position)	Manager; Planner reads
Suggestions	`/suggestions`	**Requests** (Suggestions)	Member; Manager decides
Shortlist	`/shortlist/:lineId`	**Requests** (`shortlist`, `selectCandidates`)	requesting Manager
Approvals	`/approvals`	**Allocation** (outstanding), **Requests** (`decideApproval`)	lending Manager
Inventory	`/inventory`, `/inventory/resources/:id`	**Inventory**, **Allocation** (Holds timeline, free for a window)	everyone; Resource Owner edits own
Dashboard	`/dashboard`	**Dashboard**	by role
Statement	`/statement`	**Ledger** (Statement; mid-Period Grant)	Manager (own); Planner
Settings	`/settings`	**Settings**; escalated Requests from **Requests**	Capacity Planner
Escalations	`/escalations`	**Requests** (`escalations`, `decideApproval`)	Capacity Planner
Administration	`/admin/members`, `/admin/teams`, `/admin/skills`, `/admin/types`	**Members**, **Inventory** (Resource Types)	Administrator
Profile	`/profile`, `/profile/:memberId`	**Members**, **Allocation** (Loans)	self; own Manager

Required interfaces. The REST API only (Section 7), described by the OpenAPI document at `/api/docs`.

Data owned. None on the server; in the browser, the in-memory query cache (cleared on sign-out) and the `HttpOnly` session cookie.

Invariants.

- The Web client never computes a Quote, a Balance, eligibility, or a state transition; it shows what the API returns, which is already scoped to the signed-in Member.
- Every write is an API call carrying the CSRF header; hiding a menu item is never the only guard.

Failure behavior. A `RuleViolation` (409 or 422) is shown at the field or line concerned, worded from a message table keyed by `code`. A 403 shows the "not allowed" page, a 401 returns to sign-in, a 503 asks the user to try again.

5 Runtime View

This section shows the modules at run time: the locking discipline that keeps allocation correct under concurrent use, the state machines, and seven scenarios traced through the provided interfaces of Section 4.

5.1 Transactions, locks, and the correctness guarantee

Every user or job action is one call to one `@Transactional` application-service method, which is one database transaction; modules call each other's services inside it, so a submission that places Holds, sets credits aside, writes history, and notifies lending Managers happens completely or not at all (**QA-2**, **CO-4**). No method sets an isolation level, so all run at PostgreSQL's default, READ COMMITTED, and `spring.jpa.open-in-view` is off. The correctness guarantee (**QA-1**, ADR-0006) is one sentence:

> *Every allocation decision runs in one PostgreSQL READ COMMITTED transaction that first locks the row of each Resource Type it touches, in ascending id order, so checks and writes for the same type never interleave; an exclusion constraint additionally makes it impossible for two Held or Allocated occurrences on the same Exclusive instance to overlap.*

Lock order. An allocating method locks before it reads any capacity, always in this order: (1) through `AllocationLock.lockTypes`, the `resource_type` row of every type its Request's lines name, Person included for Matched lines, one `SELECT ... FOR UPDATE` per id in ascending order; (2) in `LedgerService.holdCredits` or `commitCharges`, the Team's `account` row, where the Available Balance is checked; (3) then it writes. A type-level lock makes choosing the least-used free instance (**Exclusive.Conflict**) and inserting it one step; the Person type serializes every Matched decision, so the one-Loan check of **RC-1** cannot race; and two actions on one Request, such as a last approval and a window lapse, run one after the other.

Why READ COMMITTED is required. The lock serializes the Metered check only if the hourly sum is read by a statement that starts after the lock is granted. The local check behind this design ran two sessions on PostgreSQL 16, each adding 5 units to a pool of 8: at READ COMMITTED the second session saw the first and refused; at REPEATABLE READ with the same lock both sessions committed, 10 of 8 units. The isolation level is therefore never raised on an allocation path, and the ArchUnit test class rejects an `isolation` attribute on `@Transactional` in **Allocation** and **Requests** (Section 7).

The one retry. Three rare cases remain: a waitlist fill that locks a waiting Request's other types after holding the freed type; a second Team's `account` row (an Earning, or a Refund to a Bumped Team) written in the opposite order by another transaction; and a wait longer than `lock_timeout` (5 s on the application's connections). Spring reports these (40P01, 55P03) as `PessimisticLockingFailureException`, after the transaction has rolled back, so the retry wraps the transactional proxy: controllers and jobs call allocating services through a helper in `common` that calls the method once more in a new transaction. A second failure becomes `ConcurrencyRetryException`, HTTP 503 with a stable `code`; nothing has changed.

5.2 State machines

Request. `request.state` holds one of the seven states of Figure 6, unchanged from the SRS; only `RequestService` and `RequestLifecycleService` change it, each change with a `request_event` row (**Lifecycle.States**, **QA-3**). Submitted is entered and left inside `submit`'s transaction, so no other transaction ever reads it; it exists so that the history records the submission apart from its outcome. Withdrawn and Refused are final, and an Allocated Request stays Allocated after its windows pass.

Request Line. Each line has its own state. An Exclusive or Metered line becomes Held, Allocated, or Waiting; a Held line becomes Allocated when the Request allocates or Waiting when its Hold is released, and a Waiting line becomes Held or Allocated when capacity frees. A Matched line is AwaitingApproval, then Approved, then Loaned when the Request allocates; a Refused line returns to AwaitingApproval when a replacement is selected. Allocation rows have a smaller state: HELD, ALLOCATED, RELEASED, BUMPED, or EXPIRED, and only HELD and ALLOCATED rows count against capacity.

Approval. An `approval` is Outstanding from `ApprovalDriven.select` until the Candidate's Manager decides it, Approved or Refused, with actor, time, and reason. When the Objection Window lapses, `escalate` sets `escalated` on every Outstanding approval; the flag is not a state and stays set after the decision. Only a Capacity Planner may then decide such an approval, and later approvals on that Request are created with `escalated` set (**Matched.Escalate**).

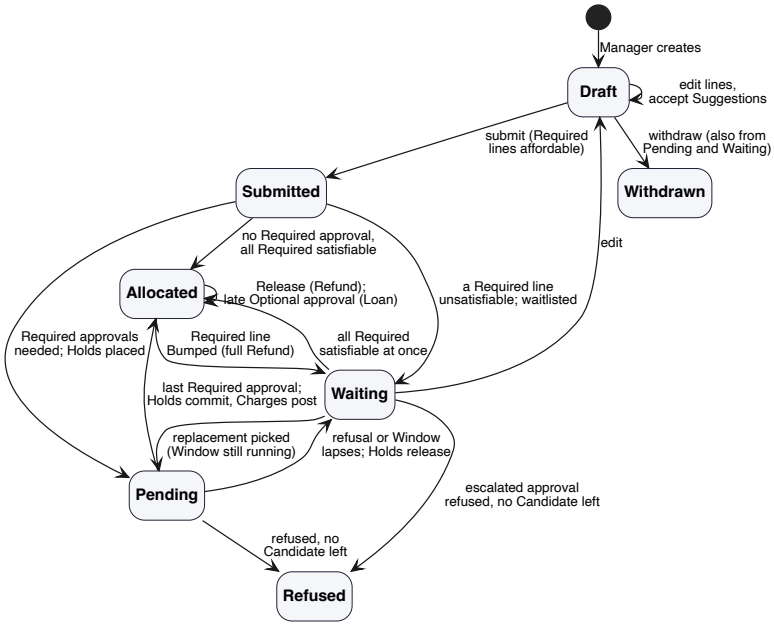


Figure 6: Request state machine, unchanged from the SRS. Withdraw is drawn once, from Draft; it is also allowed from Pending and Waiting.

5.3 Scenarios

Scenarios S1, S2, and S6 each have one sequence diagram (Figures 7, 8, and 9) with numbered messages named after the operations of Section 4 and `alt` boxes for the branches; S3, S4, S5, and S7 are given as one paragraph each.

5.3.1 S1 Submit a Request (UC-4)

1. The web client posts `/api/requests/{id}/submit`; the controller calls `RequestLifecycleService.submit`.
2. `submit` calls `LedgerService.requireAffordable`, an early check that takes no lock, with the Required lines' Quotes, locks every Quote, records `submitted_at`, and moves the Request to Submitted.
3. `AllocationLock.lockTypes` locks the lines' types; `evaluateAll` calls `Allocator.evaluate` on each line's allocator (`AllocatorRegistry.forClass`), which answers SATISFIABLE, NEEDS_APPROVAL, or UNSATISFIABLE.
4. If no Required line needs approval and every Required line is satisfiable, `allocate` Drops unfit Optional lines, writes the rows as ALLOCATED at once, and calls `commitCharges`; the Request is Allocated.
5. If a Required line needs approval, `placeHolds` calls `Allocator.hold` and `LedgerService.holdCredits` for each satisfiable Exclusive and Metered line; `buildShortlist` computes each skill-based Shortlist on read; `routeApprovals` creates named-Person approvals now and skill-based ones when the Manager picks Candidates (`selectCandidates`, `ApprovalDriven.select`), each notifying the Candidate's Manager; the Objection Window starts and the Request is Pending.
6. Otherwise `toWaiting` puts the unfit Required lines on the Waitlist and the Request is Waiting. Here and in step 4, an Optional line needing approval gets `routeApprovals` and stays open. `record` logs every change and the transaction commits.

Failure branches.

- The Required sum exceeds the Available Balance: `RuleViolation("INSUFFICIENT_BALANCE")`; nothing is locked or written and the Request stays Draft. The same code comes from `holdCredits` or `commitCharges` if a concurrent action of the Team spent the credits first.
- A Required Exclusive or Metered line is unsatisfiable: it joins the Waitlist, and the Request is Waiting, or Pending if a Required line needs approval. An unsatisfiable Optional line is never waitlisted and is Dropped at allocation.

5.3.2 S2 Last approval allocates (UC-6)

1. The lending Manager approves; `decideApproval` with `APPROVE` locks the Request's types.
2. `ApprovalDriven.decide` checks the actor (the Candidate's Manager, or after escalation a Capacity Planner),

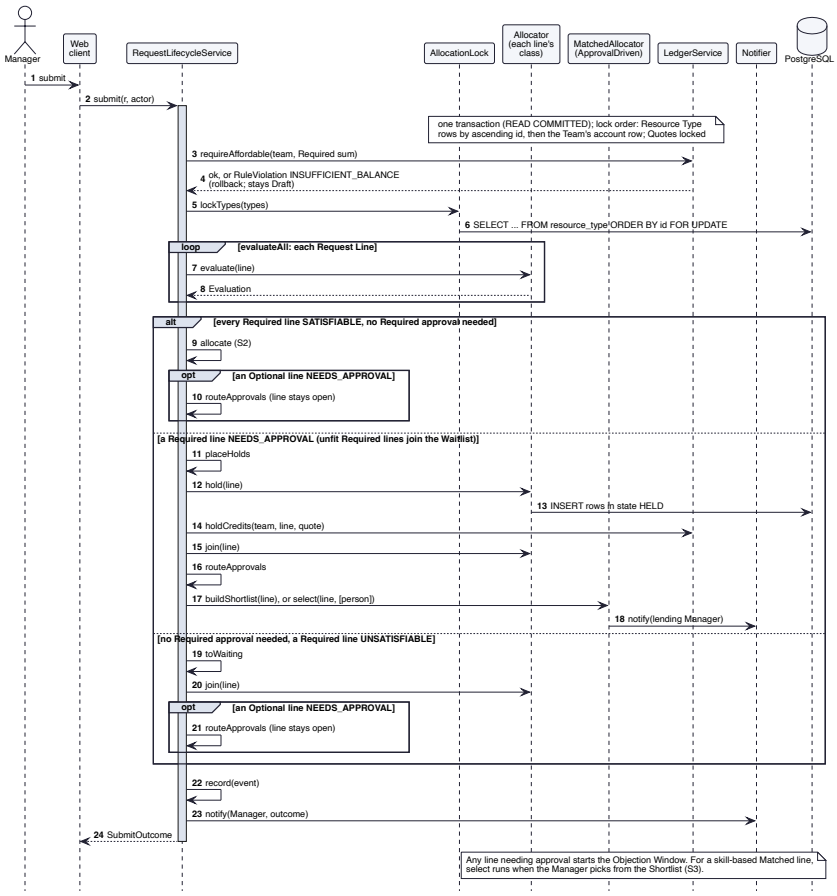


Figure 7: S1, submit a Request (UC-4). The `alt` box shows the three outcomes of evaluation (allocated at once, Pending with Holds and approvals, Waiting); the refusal for an insufficient Available Balance comes before any lock.

re-checks the Candidate's eligibility, and records Approved.

3. If a Required line is still unsatisfied, the Request stays Pending. Otherwise `allocate` re-runs `evaluateAll` under the locks; an approved Candidate who is no longer eligible (unavailable on a day, or in an overlapping Loan) makes the line UNSATISFIABLE(NOT_A_CANDIDATE).
4. Unfit or unaffordable Optional lines, and those refused with no Candidate left, are Dropped with a reason; `Allocator.commit` turns HELD rows into ALLOCATED; `LedgerService.commitCharges` turns held credits into Charges equal to the locked Quotes, granted credits first.
5. `MatchedAllocator.commit` forms the Loans of the approved Matched lines, and `LedgerService.postEarning` credits each lending Team the Day Rate times the person-days, once per Loan from `AllocationQueries.loansFor(line)`; the Request is Allocated, `record` logs it, and `Notifier` tells the Manager, the Persons, and the lending Managers.

Failure branches.

- The Candidate became ineligible before approval: `decide` refuses the approval with a `RuleViolation` whose code is `NOT_A_CANDIDATE`; the lending Manager can refuse instead (S3).
- The Balance no longer covers the Required Charges (after a rollover or an Earning correction while Pending): `toWaiting` runs instead with reason `INSUFFICIENT_BALANCE`; the approval is kept.

S3 Refusal and replacement (UC-6, UC-5). The lending Manager refuses; `decideApproval` with `REFUSE` locks the types, and `ApprovalDriven.decide` records Refused. If the refused line is Required, it is no longer satisfied, so `toWaiting` calls `Allocator.cancel` on each held line and `LedgerService.releaseCredits`, puts the Required lines on the Waitlist, and offers the freed capacity (`onCapacityFreed`). A refused Optional line releases nothing. `Notifier` asks the requesting Manager for a replacement; the Request is Waiting, or keeps its state if the refused line is Optional. The Manager picks from the Shortlist; `selectCandidates` calls `ApprovalDriven.select`, which notifies that Candidate's Manager. The window still runs, so `placeHolds` holds every satisfiable line again and the Request is Pending; `window_ends_at` is unchanged.

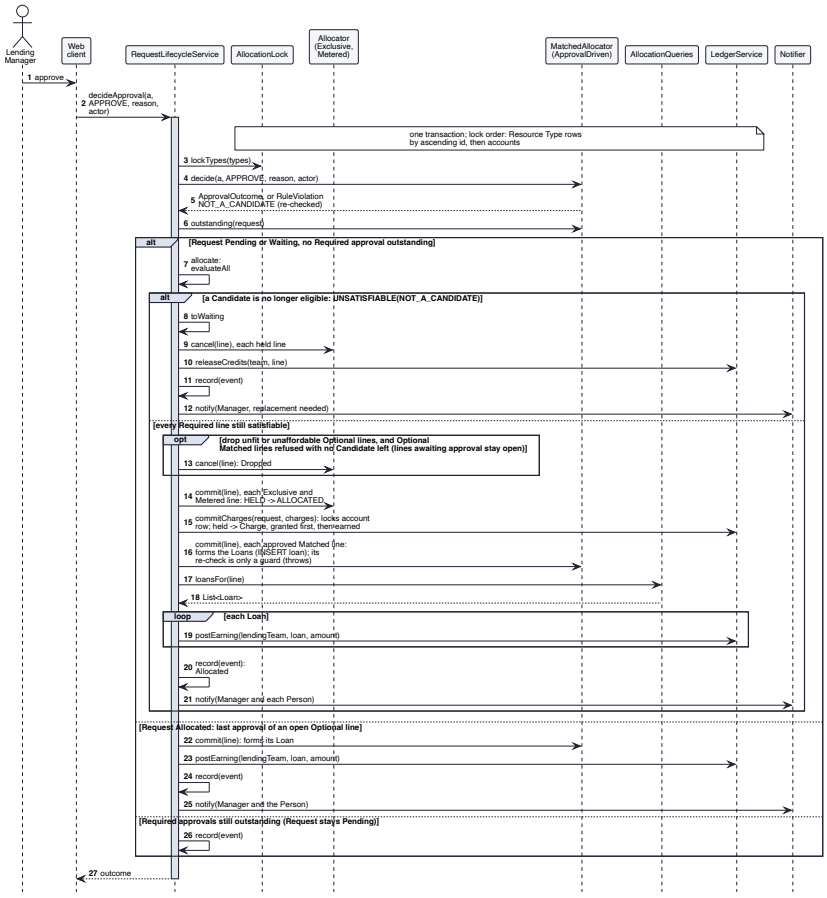


Figure 8: S2, the last Required approval allocates the Request (UC-6). Holds commit and Charges, Loans, and Earnings post in one transaction; the `alt` box shows a Candidate who is no longer eligible.

S4 Objection Window lapses (UC-7). Every 60 s `DueWorkJob.run` calls `lapsedWindows(now)`, which returns the Requests, Pending, Waiting, or Allocated, whose `window_ends_at` has passed with an approval still Outstanding and not yet escalated, and calls `onWindowLapse` for each, one transaction per Request. For a Pending Request, `toWaiting` releases every Hold (rows HELD to EXPIRED) and its held credits, waitlists the Required lines, and offers the capacity; the Request is Waiting. A Waiting or Allocated Request skips this step. `ApprovalDriven.escalate` sets `escalated` on every Outstanding approval, and `Notifier` tells the Manager and every Capacity Planner, found with `MemberService.withRole`. The Capacity Planner decides each from `escalations()` through `decideApproval` (S2 or S3). No Hold is placed again: the Request allocates only when every Required line fits at one moment (**Lifecycle.Waiting.Reevaluate**). The same run calls `expireWaitingLines(now)`, which takes lines whose window start has passed off the Waitlist.

S5 Priority line Bumps a standard line (UC-8). During `submit` (S1), `ExclusiveAllocator.evaluate` answers UNSATISFIABLE for a Priority line. The lifecycle gets `Bumpable` from `AllocatorRegistry.capability` and calls `planBump`: the instance needing the fewest Bumps of standard lines, allocated or on Hold, preferring the latest-submitted, with none starting within the bump cutoff. `applyBump` marks those occurrences BUMPED, flushes, and inserts the Priority occurrences as HELD; the exclusion constraint checks the insert. For a Bumped line that was Allocated, `LedgerService.refund` posts the full Charge back to the Bumped Team; a line that was only on Hold had no Charge, so `LedgerService.releaseCredits` releases its held credits instead. `Waitlistable.join` puts each Bumped Required line at the head of its Waitlist (a Bumped Optional line is Dropped instead); its Request moves to Waiting (other lines keep their allocations; a Pending Request releases its other Holds). `Notifier` tells the Bumped Manager why; `record` logs the Bump on both lines; the submission continues as in S1. A blocking line starts inside the bump cutoff, or only Priority lines block: no plan; the Required Priority line joins the Waitlist ahead of all standard lines.

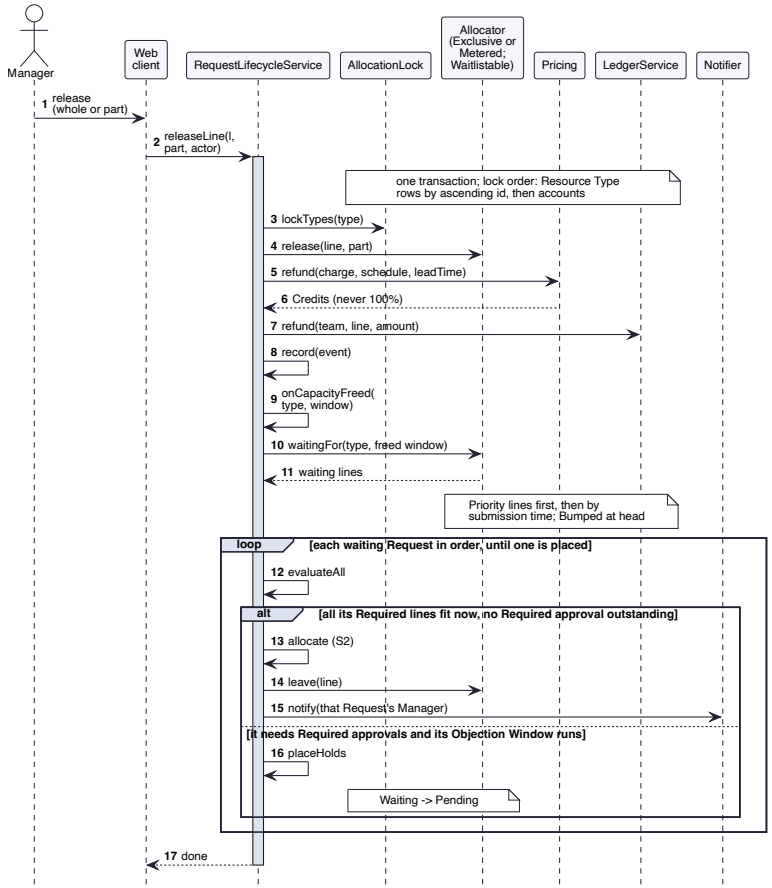


Figure 9: S6, Release and waitlist fill (UC-9, UC-10). The Refund follows the Refund Schedule, and the freed capacity is offered to the Waitlist in the same transaction; the `alt` box places Holds for a waiting Request that still needs approvals.

5.3.3 S6 Release and waitlist fill (UC-9, UC-10)

1. The Manager Releases a line or part of it; `releaseLine(l, part, ...)` locks its type, and `Allocator.release` frees that part while the rest keeps its instance and prices (**Release.Line**).
2. `Pricing.refund` applies the Refund Schedule step for the lead time to the part's start; `LedgerService.refund` posts it to the granted and earned parts in the Charge's proportions (**Release.Refund**).
3. `onCapacityFreed` asks `Waitlistable.waitingFor` for the waiting lines in order: Bumped, then Priority, then by submission time.
4. For each waiting Request in turn, the lifecycle locks its other types and checks every Required line. The first that fits is allocated (`allocate`), or, if it still needs approvals and its window runs, held (`placeHolds`) and Pending.
5. `Notifier` tells both Managers; `record` logs both histories.

Failure branches.

- The part was already consumed: `RuleViolation`; nothing changes.
- No waiting Request fits, or one cannot afford its locked Quote, or its window lapsed while it still needs approvals: it keeps its place and the next is tried; capacity left over shows as idle.

S7 Period rollover (UC-11). `PeriodRolloverJob.run` fires daily at 00:05 and acts only when a Period boundary has passed; `catchUp` runs the same check at start-up. It calls `LedgerService.rollover` with the ending and starting Periods (from `SettingsService.current()`); `rollover` locks every `account` row in ascending Team id. For each Team the remaining granted credits are the expired amount; `granted` is set to zero, and earned and held credits are untouched (**Ledger.Grant.Period**). It posts the starting Period's planned Grant from `grant_plan` (`postGrant`, when the planned amount is positive) and `Notifier` tells each Manager. No fifth Transaction kind records the expiry: `rollover` writes one `period_close` row per Team with the expired amount and closing balances; its key makes a second run a no-op.

6 Data Design

OpenBench keeps all its state in one PostgreSQL 16 database (**CO-2**, **SI-1**).

6.1 Schema by owner

Figure 10 shows the tables of the core modules grouped by owning module, and the list below gives every table and view by its owner with its purpose. Conventions: primary keys are `bigint GENERATED ALWAYS AS IDENTITY` (join tables and per-Team or per-type settings use their natural keys); instants are `timestamptz`, mapped to `Instant`, and windows are half-open, so back-to-back windows do not overlap; Loan, availability, and Period days are `date`, inclusive at both ends; credits are `numeric(12,2)`; every row has `created_at` and `created_by` (null when a job writes it); states and kinds are `text` with a CHECK on the allowed values. A foreign key may cross a module boundary as a read-only reference (`request_line.type_id` references `resource_type`), but only the owning module writes a table: its repositories are package-private, and the ArchUnit test checks it (Section 2).

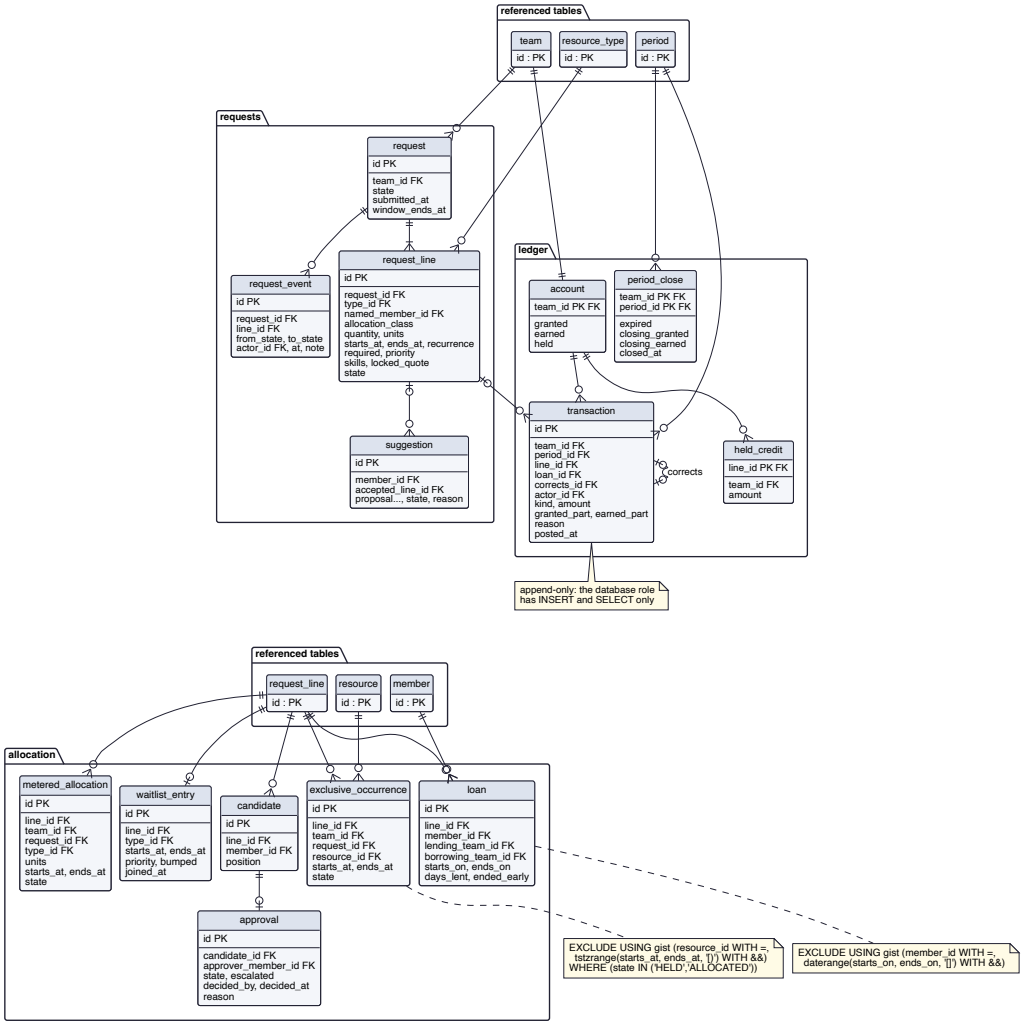


Figure 10: Database schema, core modules: above, the tables owned by **Requests** and **Ledger**; below, those owned by **Allocation**. Foundation tables they reference appear as stubs; a key that crosses a module boundary is a read-only reference. The notes quote the two exclusion constraints and the append-only rule.

- **Members**: `team` (Team); `member` (Member, login identity); `member_role` (Granted roles); `skill` (Skill list); `member_skill` (A Member's skills); `availability_range` (Days a Member may be lent).
- **Inventory**: `resource_type` (Resource Type; the row `AllocationLock` locks); `resource_type_attribute` (Attribute definitions); `resource` (Exclusive instance or Metered unit source); `resource_attribute_value` (Attribute values).
- **Settings**: `period` (Period); `grant_plan` (Grant per Team and Period); `type_pricing` (Unit Price, bump cutoff); `price_band` (Price Band); `refund_step` (Refund Schedule step); `quota` (Quota per Team and Metered type); `planner_setting` (Single-valued settings); `setting_change` (Setting change log).
- **Requests**: `request` (Request); `request_line` (Request Line, locked Quote); `suggestion` (Suggestion); `request_event` (State history).

- **Allocation**: `exclusive_occurrence` (One occurrence on one instance); `metered_allocation` (Units over a window); `waitlist_entry` (A waiting Required line); `candidate` (Shortlist entry); `approval` (Approval of a Candidate); `loan` (Loan).
- **Ledger**: `account` (Account); `transaction` (Grant, Charge, Earning, Refund); `held_credit` (Held credits per line on Hold); `period_close` (Expired and closing credits).
- **Notifications**: `notification` (In-app notification).
- **Dashboard** (views; owns no table): `v_type_hours` (Allocated and available hours per type and day); `v_team_credits` (Credits per Team and Period by kind, expired amount); `v_waiting` (Waiting lines per type).

6.2 Constraints that carry business rules

A rule between two rows, or between one row and a fixed bound, is stated in the schema, where no code path can skip it. A rule over a sum of rows is a query run under the type lock of Section 5.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE exclusive_occurrence ADD CONSTRAINT exclusive_occurrence_no_overlap EXCLUDE USING gist
  (resource_id WITH =, tstzrange(starts_at, ends_at, '[)') WITH &&) WHERE (state IN ('HELD', 'ALLOCATED'));
ALTER TABLE loan ADD CONSTRAINT loan_one_at_a_time EXCLUDE USING gist
  (member_id WITH =, daterange(starts_on, ends_on, '[]') WITH &&);
REVOKE UPDATE, DELETE ON transaction, request_event, setting_change FROM openbench_app;
```

No overlapping allocation, no double Loan. A row with the same Resource as a live row and an overlapping window is rejected. The second constraint keeps a Person in at most one Loan at a time on every code path, with no exception for `ended_early`, which only marks that an Earning correction was posted. Only HELD and ALLOCATED rows take part in the first constraint; RELEASED, BUMPED, and EXPIRED rows stay as history and block nothing.

The append-only Ledger. Flyway connects as the schema owner; the application connects as `openbench_app`, which owns nothing. The application can insert and read Transactions but never change or delete one, so **Ledger.Transaction.Kinds** and **QA-3** hold even against a bug. A correction is a new row whose `corrects_id` names the one it corrects; the only one in Release 1, the Earning correction of an early Loan end, is the only negative amount and the only Transaction that may take a Balance below zero.

Check constraints. A Refund Schedule never reaches 100% (**Release.Refund**): `CHECK (fraction >= 0 AND fraction < 1)` on `refund_step`.

The Metered hourly sum. **Metered.Pool.Check** needs, in every hour of the window, the units of live rows on the type plus the requested units within Pool Capacity, and the Team's share within its Quota. `MeteredAllocator` runs a native query after `lockTypes`, in the same READ COMMITTED transaction, and reads Pool Capacity after the lock too. The pure class `HourlyLoad` (Section 4.5) then walks the hours in order and returns the earliest one in which `pool_used` plus the requested units exceed Pool Capacity (`POOL_FULL`) or, when the Team has a Quota on the type, `team_used` plus them exceed it (`QUOTA_EXCEEDED`); no such hour means the line fits. With zero requested units and the reduced capacity, the same query and `HourlyLoad` answer `ReductionGuard.earliestHourExceeding` (**Inventory.Resource.Add**).

6.3 Migrations and the test database

Spring Boot runs Flyway at start-up, before Hibernate, because the `EntityManagerFactory` depends on database initialization. `spring.jpa.hibernate.ddl-auto=validate` then checks every entity against the migrated tables and stops start-up on a mismatch; Hibernate never creates a table, and the migrations alone own the extension, the exclusion constraints, the partial indexes, and the grants. The SRS first required tests on H2 in memory with the same migrations. H2 cannot run them: in PostgreSQL mode it rejects `CREATE EXTENSION`, has no range types or `EXCLUDE`, and cannot parse the hourly sum's interval arithmetic. SRS 0.3 therefore changes **CO-2** and **SI-1** to PostgreSQL 16 in a container started by the test suite (**RC-3**). Integration tests extend one base class with `@Testcontainers` and a `PostgreSQLContainer` (`postgres:16-alpine`); `@ServiceConnection` points the data source at the container, so Flyway runs the production migrations, extension and constraints included. The concurrency test runs two allocations on two threads against one pool and asserts that exactly one succeeds; another connects as `openbench_app` (created by the container's init script) and checks that updating a Transaction is refused (Section 7). Nothing is purged in Release 1 (**QA-7**): no code path deletes a Request, a line, a Suggestion, a `request_event`, or a Transaction, and Teams, skills, Resource Types, and Resources are retired, not deleted.

7 Interfaces and Crosscutting Concerns

The browser client talks to the server only through one REST/JSON API, and every use case of the SRS can be carried out through it under the same role checks (**SI-2**, **CO-3**).

7.1 API Conventions

Paths. Every endpoint lives under `/api`; any other path returns the single-page application. Where a path needs the data of a higher module, that module's controller serves it, so no controller breaks the dependency rule of Section 2: the holder views under `/inventory` and a Member's Loans under `/members` are served by **Allocation**.

Methods and transitions. GET reads and never changes state; POST creates an object (201 with `Location`) or performs one transition; PUT replaces one value; DELETE removes a Draft line or a role. A transition is a POST to a sub-resource named by the verb of the state machine, such as POST `/api/requests/{id}/submit`, `/withdraw`, and `/reopen`, POST `/api/requests/{id}/lines/{lineId}/release`, and POST `/api/approvals/{id}/decision`; it answers 200 with the object's new state.

One path through the server. A controller method binds and validates the body, builds the caller's `Actor` from the session, calls one application-service operation (the free-Resource search alone calls two read-only queries), and maps the result to JSON. `@PreAuthorize` sits on the application-service methods, not on the controllers, so the screens and any other API client pass the same checks (**CO-3**, **SE-1**).

Representation. JSON in UTF-8 with camelCase names; enumerations as upper-case strings (`PENDING`); ids as JSON integers (the typed wrappers such as `RequestId` serialize as their number); credits as JSON numbers; instants as ISO-8601 date-times with the offset of the company time zone (`2026-10-12T09:00:00-04:00`), accepted with any offset and normalized; Loan days and Period bounds as ISO dates. A time window is `{"start", "end"}`, half-open as `TimeWindow`; a day range is `{"from", "to"}`, inclusive as `DateRange`.

Errors. Every error is an RFC 9457 problem detail (`application/problem+json`, Spring's `ProblemDetail`) with `type`, `title`, `status`, `detail` (one sentence the client shows as is), `instance` (the request path), and the extension field `code`, a stable identifier the client may branch on; a 400 adds `errors`, one entry per invalid field. Table 4 gives the status per error class.

Table 4: HTTP status per error class.

Status	Error class and typical cause	Example `code`
400 Bad Request	Validation at the controller: malformed JSON, a missing field, a value out of range	`VALIDATION_FAILED`
401 Unauthorized	Unauthenticated: no session, an expired session, wrong credentials at login	`UNAUTHENTICATED`, `BAD_CREDENTIALS`
403 Forbidden	A missing role (`@PreAuthorize`), a missing or wrong CSRF token, or `ForbiddenException`	`FORBIDDEN`, `CSRF_REJECTED`
404 Not Found	`NotFoundException`, including any object outside the caller's scope	`NOT_FOUND`
409 Conflict	`RuleViolation` on a state conflict: the object's state does not allow the action; or an exclusion-constraint violation (SQLSTATE 23P01), logged as a bug	`WRONG_STATE`, `ALREADY_DECIDED`, `ALLOCATION_CONFLICT`
422 Unprocessable Content	`RuleViolation` on a business rule	`INSUFFICIENT_BALANCE`, `NOT_A_CANDIDATE`
503 Service Unavailable	`ConcurrencyRetryException`: a lock wait or deadlock failed twice; `Retry-After: 1`	`CONCURRENCY_RETRY`
500 Internal Server Error	Anything else; logged with its stack trace, never returned in the body	`INTERNAL`

API documentation. springdoc serves the OpenAPI 3 document at `/api/docs` and a browsable page at `/api/docs/ui` to signed-in Members.

7.2 Authentication and Sessions

Authentication is Spring Security form login with a server-side HTTP session (**CO-6**, **SE-2**). **Login and logout.** POST `/api/login` takes the form fields `username` and `password` and answers with JSON instead of a redirect: 200 with the body of GET `/api/me`, or a 401 that does not say whether the username exists. **Identity.** GET `/api/me` returns the Member's id, name, Team, and roles. **Session cookie.** `HttpOnly; Secure; SameSite=Lax`. Sessions live in the memory of the one process and expire after 12 hours without a request. **CSRF.** Cookie to

header: the token travels in the cookie `XSRF-TOKEN`, which script may read, and the client copies it into the header `X-XSRF-TOKEN` on every POST, PUT, and DELETE, the login included. **Passwords and transport.** bcrypt through Spring's delegating password encoder, which stores the algorithm with the hash so it can be upgraded without a reset; no password is logged or returned. HTTPS only (**CI-1**): TLS ends at nginx or the embedded Tomcat, plain HTTP only redirects, responses carry Strict-Transport-Security, and behind nginx the forwarded headers are trusted so redirects and the `Secure` cookie use the external scheme. **Sign-in log.** Every authentication attempt, successful or not, is logged with username, remote address, and outcome (**SE-2**).

7.3 Endpoints by Module

Each module serves its own group of endpoints, so every use case maps to endpoints (**SI-2**): the session (`/login`, `/logout`, `/me`); **Members** (`/teams`, `/admin`, `/skills`, `/members`, with a Member's Loans served by **Allocation**); **Inventory** (`/inventory`, with the holder timeline and the free-for-a-window search served by **Allocation**); **Settings** (`/settings`); **Requests** (`/requests` and `/loans` for Drafts and every transition, `/suggestions`, `/approvals`, `/escalations`); **Ledger** (`/ledger`); **Dashboard** (`/dashboard`); and **Notifications** (`/notifications`). Roles are granted authorities, and every signed-in Member holds `MEMBER`; the longer edition lists every endpoint with its method, roles, and operation. Operations only other modules call, such as `holdCredits`, `isAvailable`, `onWindowLapse`, and `rollover`, have no endpoint.

The key of PUT `/api/settings/{key}` names one setting and the body carries its new value: `day_rate`, `priority_premium`, `objection_window_working_days`, `period_months`, `grant.{teamId}.{periodId}`, `quota.{teamId}.{typeId}`, `unit_price.{typeId}`, `bump_cutoff.{typeId}`, `price_bands.{typeId}`, or `refund_steps.{typeId}`. UC-10, filling a freed slot from the Waitlist, has no endpoint of its own: it runs inside `releaseLine`, `withdraw`, `decideApproval`, and the jobs, wherever capacity frees.

7.4 Authorization and Data Scoping

Every request passes two checks: a role check and a scope (**SE-1**, **SE-4**).

Roles. Each `member_role` row becomes a Spring Security granted authority, `MANAGER`, `RESOURCE_OWNER`, `PLANNER`, or `ADMIN`, and every signed-in Member also holds `MEMBER`. Each application-service method a controller calls carries `@PreAuthorize` with the roles of its endpoint group (Section 7), so the check runs on the server in the same place whether the call comes from a screen or a script (**CO-6**, **CO-3**); an ArchUnit rule fails the build if a controller reaches a service method without one.

Scope. Which rows a role may see is decided inside the query, by the caller's Team, own id, or owned Resources, never by loading rows and filtering them afterward. The controller passes the caller as an `Actor` (Member id, Team id, roles), and repositories take its fields as parameters. A row outside the scope is not found, so the caller gets 404 and learns nothing about it.

```
// RequestRepository: the scope is part of the query
@Query("select r from Request r where r.id = :id and (r.teamId = :team or :planner = true)")
Optional<Request> findVisible(RequestId id, TeamId team, boolean planner);
// RequestLifecycleService.detail
Request r = requests.findVisible(id, actor.teamId(), actor.has(PLANNER)).orElseThrow(NotFoundException::new);
```

7.5 Audit Trail

Three records keep the history, each written by its owning module in the same transaction as the change it records, so a change and its record commit together or not at all (**QA-3**, **Lifecycle.States**). `RequestLifecycleService.record` writes a `request_event` row (Request, line, from-state, to-state, actor, time, note) for every state change of a Request or a Request Line. `SettingsService.update` writes a `setting_change` row (key, old value, new value, actor, time) for every Capacity Planner change (**Admin.Member.Create**, **Planner.Settings**). Rows of `transaction` are append-only (**Ledger.Transaction.Kinds**): the application's database role may insert and select them but not update or delete them, so a correction is a new row naming the one it corrects.

7.6 Notifications

Written in the caller's transaction. The module that causes the event calls `Notifier.notify(to, kind, text, link)` inside its own transaction, and `Notifier` inserts the row and hands it to each `Delivery`; `InAppDelivery`, the only one in Release 1, does nothing more, so a rolled-back action leaves no notification and nothing is sent after commit (**CI-2**). The client polls GET `/api/notifications` every 30 seconds and shows the unread count; `link` is the client route to open, such as `/requests/42`. Recipients are resolved through **Members**: a Team's Manager, or every Member holding `PLANNER`.

7.7 Error Taxonomy and Validation

Four exception classes in `common` carry every refusal, and one `@RestControllerAdvice` in `common` turns them into the problem details of Table 4.

- `NotFoundException`: the object does not exist or lies outside the caller's scope (404).
- `ForbiddenException`: the caller sees the object but may not act on it, such as a lending Manager ending a Loan only the borrowing Manager may end (403).
- `RuleViolation(code, message)`: a business rule refused the action. Codes that describe the object's current state (`WRONG_STATE`, `ALREADY_DECIDED`, `WINDOW_LAPSED`, `SUGGESTION_CLOSED`, `NOT_ALLOCATED`, `PERIOD_STARTED`) map to 409, all others to 422.
- `ConcurrencyRetryException`: an allocation transaction failed twice on a deadlock or lock timeout, which Spring reports as `PessimisticLockingFailureException` (503).

Where validation happens. At the edge, bean validation (`@Valid` on request bodies) checks shape only: presence, types, ranges such as a positive quantity, formats, an end after a start; it reads no data. The application services and domain objects check every business rule: a window in the future, a type not retired, affordability, the Candidate count and eligibility, capacity, the allowed transitions.

7.8 Configuration and Time

Deployment configuration comes from environment variables through Spring Boot's relaxed binding, so the one Docker image runs unchanged everywhere and no secret lives in the image or the repository (**UD-2**). Fixed settings (Flyway on, schema validation, a two-thread scheduler) ship in the jar. Time is read only from one injected `java.time.Clock` bean set to the company zone, never from the no-argument `now()` methods, which an ArchUnit rule forbids; tests swap in a clock they move by hand to lapse an Objection Window or cross a Period boundary. All windows, Price Bands, and Periods are in the one company time zone (**LI-1**).

7.9 Testing Strategy

Pure rules are tested as plain objects; everything the database guarantees is tested on PostgreSQL 16 in a container started by the test suite with the production migrations, never an in-memory substitute (**RC-3**, **CO-2**). `mvn verify` runs every level below except usability; the container tests need Docker.

- **Unit: pricing and time.** JUnit 5 with a fixed `Clock`, no Spring or database: `Pricing.quote` (band edges, premium, a daylight-saving day) and `Pricing.refund` (each step, never 100%) (**Ledger.Price.Formula**, **Release.Refund**).
- **Unit: rules.** The Shortlist filter (skills, availability, overlapping Loan, own Team, inactive) and order; the Request and line state machines, every allowed transition and every other one refused (**Matched.Shortlist.Build**, **Lifecycle.States**).
- **Integration.** Spring Boot tests on PostgreSQL 16 in Testcontainers with the Flyway migrations: the hourly-sum SQL with Pool Capacity and Quota, both exclusion constraints, the append-only permissions on `transaction`, lapse and rollover with a moved clock, scoping (another Team's Request is 404), the problem-detail mapping (**QA-1**, **QA-3**, **SE-4**, **PE-4**).
- **Concurrency.** Two threads released together by a latch, each in its own transaction. Metered: both ask for the last units of one hour; exactly one is Allocated, the other Waiting, and no hour exceeds Pool Capacity (**QA-1**, **CO-4**).
- **Architecture.** One ArchUnit test class: the dependency rule of Section 2 (layer order, no cycles, entities and repositories private to their module, `ReductionGuard` the only upward reference); `@PreAuthorize` on every service method a controller calls; no no-argument `now()`; no `isolation` attribute on `@Transactional` in **Allocation** and **Requests** (**CO-7**, **QA-6**, **SE-1**).
- **End-to-end.** One test over HTTP with a real session and CSRF token: a Manager submits the three-line Request (Matched, Exclusive, Metered) and selects a Candidate; the lending Manager approves; the Request is Allocated and the Charges and the Earning appear on both Teams' Statements (**Lifecycle.Allocate**, **QA-2**).
- **Front end.** Vitest and React Testing Library at the route level, the API mocked through the generated types: each route renders for the right roles, sends the right call, and shows the problem `detail` on a refusal (**UI-1**, **UI-2**, **UI-3**).
- **Usability.** Before the final demonstration, at least three untrained people from outside the project team each draft and submit a three-line Request in under five minutes (**QA-5**).

Each test's display name carries the identifiers it covers, so the traceability of Appendix A can be checked against the test report (**QA-7**).

8 Maintenance and Evolution

Release 1 is fixed. Stretch items are built only if time allows, and Future items have no requirements yet. For each of them, and for the operational changes a successful deployment would bring, the design names the one place it lands, so none of them is a precondition for Release 1 and none changes the dependency direction of Section 2.

8.1 Extension Points

Table 5 covers every Stretch and Future item of the SRS scope table plus two operational changes. Most rows add a class or a column behind an interface that already exists; the longer edition also lists, for each row, what does not change.

Table 5: Extension points for the Stretch, Future, and operational items.

Item	Tier	Where it lands	What changes
Service tiers and preemption on Metered pools (**Metered.Tier.Define**, **Metered.Tier.Bump**)	Stretch	**Allocation**, **Ledger**, **Settings**, **Requests**	`MeteredAllocator` also implements `Bumpable`; a tier premium multiplies the Unit Price in `Pricing.quote` as the priority premium does; tiers per type become settings; `request_line` gains a tier. The lifecycle, which already asks `AllocatorRegistry` for `Bumpable`, does not change
Demand surcharge (**Ledger.Surcharge**)	Stretch	**Requests**, **Ledger**, **Settings**	**Requests** reads the requested window's fullness from `AllocationQueries` and passes it into `Pricing.quote` as an extra input; one multiplier inside `quote`. `Pricing` stays pure and **Ledger** never calls **Allocation**
Actual usage recording and comparison (**Metered.Usage.Record**, **Metered.Usage.Compare**, **Dashboard.Usage**)	Stretch	**Allocation**, **Dashboard**	a `used_units` column on `metered_allocation`, written by a new Resource Owner operation in **Allocation**; one **Dashboard** view comparing requested and used units. **Ledger** is untouched
Email notifications (**CI-2**)	Stretch	**Notifications**, **Settings**	`EmailDelivery` implements `Delivery` and sends after commit; SMTP settings become settings the Administrator edits. No caller of `Notifier` changes
Demand analytics on declined Suggestions	Future	**Dashboard**	one view over Declined `suggestion` rows per Period; Suggestions keep their original ask and are never deleted
Person consent (lifts **LI-4**)	Future	**Allocation**	`MatchedAllocator.select` first creates a consent approval addressed to the Person, then the Manager's; `approval` gains a kind
Skill-based Day Rate	Future	**Settings**, **Ledger**	a rate per skill in Settings with the one Day Rate as the default; a rate lookup next to `Pricing` computes a Loan's Earning from its skills
Team rank priority	Future	**Allocation**, **Settings**	a rank per Team set by the Capacity Planner; one comparator in the Bump plan of `ExclusiveAllocator` consults it
Inter-Team transfers	Future	**Ledger**	a fifth Transaction kind, Transfer, and one operation posting a pair of Transactions between two Teams with a reason
Forecasting accuracy reward	Future	**Ledger**, **Jobs**	at rollover, a Grant computed from requested versus used units (needs usage recording)
Usage feeds	Future	**Allocation**	an adapter fills `used_units` from a scheduler or license server instead of the Resource Owner
Auctions	Future	**Allocation**	a new Allocation Class whose allocator defers, as `ApprovalDriven` does, until bidding closes, with the winning bid as the Quote
SSO (lifts **LI-3**)	Future	`security`	a Spring Security authentication provider (OpenID Connect or SAML) mapping the identity provider's subject to `member.username`
Calendar integration (lifts **AS-4**)	Future	**Members**	an adapter behind `isAvailable` and `availableWithSkills` that also removes calendar busy days
HR integration	Future	**Members**	an import adapter that creates, moves, and deactivates Members and Teams from the HR system
Cluster-scheduler integration	Future	**Inventory**, **Jobs**	an adapter sets Resource units through `setUnits` and its guard; a polling step in **Jobs** pushes committed Metered allocations to the scheduler

continued on the next page

Item	Tier	Where it lands	What changes
A second application instance	Operational	**Jobs**, `security`	a job lock table each job claims with a conditional update before it runs; sticky sessions at nginx or a shared session store. Allocation correctness already holds across instances, since type locks and exclusion constraints live in the database
A new Allocation Class (**QA-6**, **CO-7**)	Operational	**Allocation**	one `Allocator` bean and the capability interfaces it needs; its name in the `AllocationClass` enumeration and the check on `resource_type`. **Requests**, **Ledger**, and the existing allocators do not change

One dependency added during design. Ledger calls **Notifications** to tell each Manager about Period rollover and mid-Period Grants; **Notifications** sits in the bottom layer, so the dependency rule still holds and the ArchUnit test allows it.

8.2 Assumptions That Would Break the Design

- **One company in one time zone** (**LI-1**): Price Bands, Period boundaries, person-days, the rollover time, and the API's offsets all use one configured zone; a zone per Team or Resource would change Price Band lookup, Periods, and rollover.
- **One server instance, no high availability** (**LI-1**, **QA-4**): each job runs once because one process runs it, and sessions live in memory; a second instance needs a job lock table and shared sessions (Table 5), and allocation correctness is unaffected.
- **Hour granularity for Metered pricing and checks; day granularity for Loans**: finer windows multiply the hourly check's buckets and change Quote arithmetic; half-day Loans change person-days, the Day Rate, and the date-range constraint.
- **The Capacity Planner sets Grants before the first Period** (**AS-1**): without Grants only Matched lines can be submitted (`INSUFFICIENT_BALANCE`); the design invents no default Grant.
- **Loans are recorded honestly** (**AS-3**): the Earning posts when the Loan forms and is corrected only by an early end; confirming the work would need a record of days actually worked.
- **The scale of PE-1**: the Shortlist is built on read, hourly sums are computed by query, and the Waitlist is walked inside the releasing transaction; at a hundred times the scale, stored hourly totals per type, a precomputed Shortlist, and a Waitlist fill moved to `DueWorkJob`.

8.3 Risks

Likelihood and impact are low, medium, or high, as in the SRS.

- **Lock-order discipline breaks** (medium; high: an oversold Metered pool, or deadlocks): `AllocationLock.lockTypes` takes a sorted set, so the order holds by construction; every path that consumes or reduces capacity runs through `RequestLifecycleService` or `ReductionGuard`, which lock first; the exclusion constraints backstop Exclusive instances and Loans; `lock_timeout` turns a stuck wait into a 503. Metered pools have no constraint backstop, so the Metered race test and code review guard them.
- **The exclusion constraints depend on the `btree_gist` extension** (low; high: the server does not start): the first Flyway migration creates the extension, so a missing privilege fails at start-up and not at the first Request; every test run proves it on PostgreSQL 16.
- **Waitlist walk cost grows with the Waitlist** (low at **PE-1**; medium): `waitingFor(type, freed)` reads only entries whose window overlaps the freed one, through an index on type and start; the walk stops at the first Request that allocates; if needed, the fill moves to `DueWorkJob` and still meets **PE-4**.
- **The single polling job stalls or fails** (low; medium: lapses and expiries come late): one transaction per Request, so one failure does not stop the rest; due work is found by query, so a missed run or a restart is caught up on the next run.
- **The front end drifts from the API** (medium; medium): client types are generated from the OpenAPI document on every build, so a changed endpoint fails the TypeScript compile; Quotes and rules live only on the server.
- **Four students, one semester** (**RI-5**; high; high): Release 1 is fixed and every Stretch and Future item is an extension point, never a precondition; modules are built bottom-up in the layer order against the interfaces of Section 4, each testable alone; Stretch work starts only after the end-to-end test of the three-line Request passes.

9 References

1. J. Nana, B. Lujan, F. Turopov, Y. Shnazbayev, *Software Requirements Specification for OpenBench*, version 0.4, 2026-11-07; short edition `docs/srs/OpenBench-SRS-short.pdf` (the identifiers used here) and full edition `docs/srs/OpenBench-SRS.pdf`, project repository.
2. J. Nana, B. Lujan, F. Turopov, *OpenBench: A Policy-Pluggable Internal Resource Marketplace*, CSDS 393 informal project proposal, 2026.
3. Project vocabulary file `CONTEXT.md` and architecture decision records ADR-0001 to ADR-0007 in `docs/adr/`, project repository, 2026.
4. Spring Boot reference documentation, version 3. https://docs.spring.io/spring-boot/reference/
5. Spring Security reference documentation. https://docs.spring.io/spring-security/reference/
6. Spring Data JPA reference documentation, including locking. https://docs.spring.io/spring-data/jpa/reference/
7. Hibernate ORM 6.6 user guide. https://docs.hibernate.org/orm/6.6/userguide/html_single/
8. PostgreSQL 16 documentation, https://www.postgresql.org/docs/16/, in particular the chapters on range types (`rangetypes.html`), exclusion constraints (`ddl-constraints.html`), the `btree_gist` extension (`btree-gist.html`), explicit locking (`explicit-locking.html`), and transaction isolation (`transaction-iso.html`).
9. Flyway documentation. https://documentation.red-gate.com/fd
10. Testcontainers for Java, PostgreSQL module. https://java.testcontainers.org/modules/databases/postgres/
11. React documentation. https://react.dev/
12. Vite guide. https://vite.dev/guide/
13. TanStack Query documentation. https://tanstack.com/query/latest
14. M. Nottingham, E. Wilde, S. Dalal, *Problem Details for HTTP APIs*, RFC 9457, IETF, 2023. https://www.rfc-editor.org/rfc/rfc9457
15. Object Management Group, *Unified Modeling Language*, version 2.5.1, 2017. https://www.omg.org/spec/UML/2.5.1/
16. PlantUML language reference. https://plantuml.com/
17. ArchUnit user guide. https://www.archunit.org/userguide/html/000_Index.html
18. springdoc-openapi documentation. https://springdoc.org/

A Traceability

Each row traces one identifier of the SRS version 0.4 short edition to the module or modules that satisfy it, the first named owning the requirement, and to the sections of this document that design it; Stretch requirements point at their extension point in Section 8. Every Release 1 requirement maps to at least one module; the requirement wording is in the SRS.

Table 6: Traceability from SRS identifiers to modules and sections.

Identifier	Module(s)	Section
Functional requirements, Release 1		
Inventory.Type.Define	**Inventory**	4.2, 6
Inventory.Resource.Add	**Inventory**, **Allocation**	4.2, 4.5
Inventory.View.Holder	**Allocation**, **Inventory**	4.5, 4.2, 7
Profile.Skills	**Members**	4.1
Profile.Availability	**Members**, **Allocation**	4.1, 4.5, 6
Suggest.Create	**Requests**	4.7
Suggest.Accept.Existing	**Requests**, **Notifications**	4.7, 4.4
Draft.Create	**Requests**	4.7, 7
Draft.Line.Add	**Requests**	4.7
Draft.Line.Required	**Requests**	4.7
Draft.Quote.Line	**Requests**, **Ledger**	4.7, 4.6
Draft.Submit.Lock	**Requests**	4.7, 5
Matched.Shortlist.Build	**Allocation**, **Members**	4.5, 5
Matched.Approve.Route	**Allocation**, **Requests**, **Notifications**	4.5, 5
Matched.Refuse.Replace	**Requests**, **Allocation**	4.7, 5
Matched.Escalate	**Requests**, **Allocation**	4.7, 5
Matched.Loan.Form	**Allocation**, **Ledger**, **Notifications**	4.5, 5, 6
Exclusive.Conflict	**Allocation**	4.5, 5, 6

Traceability (continued)

Identifier	Module(s)	Section
Exclusive.Waitlist.Join	**Allocation**, **Requests**	4.5, 5
Exclusive.Priority.Bump	**Allocation**	4.5, 5
Exclusive.Bump.Effect	**Allocation**, **Requests**, **Ledger**, **Notifications**	4.5, 5
Metered.Line.Form	**Allocation**, **Requests**, **Ledger**	4.5, 4.7
Metered.Pool.Check	**Allocation**	4.5, 5, 6
Lifecycle.States	**Requests**	4.7, 5
Lifecycle.Evaluate	**Requests**, **Allocation**	4.7, 5
Lifecycle.Hold.Place	**Requests**, **Allocation**, **Ledger**	4.7, 4.5, 5
Lifecycle.Window.Start	**Requests**, **Jobs**	4.7, 4.9, 5
Lifecycle.Allocate	**Requests**, **Allocation**, **Ledger**, **Notifications**	4.7, 5
Lifecycle.Waiting.Reevaluate	**Requests**, **Allocation**	4.7, 5
Release.Line	**Requests**, **Allocation**	4.7, 4.5, 5
Release.Refund	**Ledger**, **Requests**	4.6, 5
Release.Loan.Prorate	**Requests**, **Allocation**, **Ledger**	4.7, 4.6
Ledger.Account	**Ledger**	4.6, 6
Ledger.Transaction.Kinds	**Ledger**	4.6, 6
Ledger.Grant.Period	**Ledger**, **Jobs**	4.6, 4.9, 5
Ledger.Price.Formula	**Ledger**, **Settings**	4.6, 4.3
Ledger.Earning.Post	**Ledger**	4.6
Dashboard.Utilization	**Dashboard**	4.8, 6
Dashboard.Credits	**Dashboard**	4.8, 7
Admin.Member.Create	**Members**, **Settings**	4.1, 4.3, 7
Planner.Settings	**Settings**, **Requests**	4.3, 4.7
Functional requirements, Stretch		
Metered.Tier.Define	**Settings**, **Ledger** (tier premium in `Pricing`)	8, service tiers
Metered.Tier.Bump	**Allocation** (`Bumpable` on `MeteredAllocator`)	8, service tiers
Metered.Usage.Record	**Allocation** (usage beside `metered_allocation`)	8
Metered.Usage.Compare	**Dashboard** (one more view)	8
Ledger.Surcharge	**Ledger** (multiplier in `Pricing`), **Settings**	8, demand surcharge
Dashboard.Usage	**Dashboard** (one more view)	8
Constraints, interfaces, and nonfunctional requirements		
CO-1	All modules	2, 4.10
CO-2	All back-end modules	6, 7
CO-3	**Web client**	7, 4.10
CO-4	**Ledger**, **Requests**	4.6, 5
CO-5	**Jobs**	4.9, 5
CO-6	**Members**, `security` package	7, 7
CO-7	**Allocation**	4.5, 8
UI-1	**Web client**, **Requests**	4.10, 4.7
UI-2	**Web client**, **Allocation**	4.10, 4.5
UI-3	**Web client**	4.10
SI-1	All back-end modules	6
SI-2	All back-end modules, **Web client**	7
CI-1	All modules (deployment)	2, 7
CI-2	**Notifications**	4.4, 7
PE-1	All modules	2, 6
PE-2	All modules	2, 7
PE-3	**Allocation**	4.5
PE-4	**Jobs**	4.9, 5
SE-1	All modules, `security` package	7, 7
SE-2	**Members**, `security` package	7, 7
SE-3	**Members**	4.1, 6
SE-4	All modules	7
QA-1	**Allocation**	4.5, 5, 6
QA-2	**Requests**, **Ledger**	4.7, 5
QA-3	**Ledger**, **Requests**, **Settings**	7, 6
QA-4	**Web client**	4.10, 2
QA-5	**Web client**	4.10
QA-6	**Allocation**, **Inventory**	4.5, 8
QA-7	All modules	6, 7
User documentation, assumptions, and dependencies		
UD-1	**Web client**	4.10
UD-2	**Members**, **Settings**	7, 8
UD-3	**Web client**	4.10
AS-1	**Settings**, **Ledger**	4.3, 8
AS-2	**Inventory**	4.2
AS-3	**Allocation**	4.5, 8
AS-4	**Members**	4.1, 8

Traceability (continued)

Identifier	Module(s)	Section
DE-1	All modules	2
DE-2	All modules (build)	2

B Glossary

The domain vocabulary below is generated at build time from the project's shared vocabulary file and is identical to the glossary of the SRS; the design terms after it are specific to this document.

People and roles

Member Any user of the system. Belongs to exactly one Team, has a skill profile and availability, and is lendable as a Person.

Team The organizational unit that owns an Account and every Request made on its behalf. Has exactly one Manager.

Manager The Member who leads a Team. The only actor who submits Requests, and the approver for lending that Team's Members.

Resource Owner A Member responsible for the Inventory records of specific Exclusive or Metered resources. Never approves allocations.

Capacity Planner The role that sets each Team's Grant per Period, Unit Prices, the Day Rate, and Refund Schedules, and receives escalations.

Administrator The role that manages users, Teams, and Resource Type definitions. Configures the system; sets no policy.

Resources

Resource Anything that can be allocated: a Person, an Exclusive resource such as a room, or a Metered pool such as GPU hours. Every Resource has exactly one Resource Type.

Resource Type A named definition, created as data by an Administrator, that fixes a Resource's Allocation Class and attributes.

Allocation Class One of three policies a Resource Type declares, deciding how Requests for it are allocated: Matched, Exclusive, or Metered.

Matched The Allocation Class for People. Allocated by matching skills and availability, gated by the Person's Manager.

Person A Member viewed as a Matched Resource: someone with skills and availability who can be lent to another Team.

Exclusive The Allocation Class for Resources held whole over a time window, such as rooms or instruments. Allocated with conflict detection, a waitlist, and priority bumping.

Metered The Allocation Class for divisible Resources drawn against a Team quota, such as compute hours or license seats.

Pool Capacity The total units a Metered Resource Type can supply at once, such as 32 GPUs, maintained by its Resource Owner. Allocations in any hour may not exceed it.

Quota An optional per-Team cap on the units of one Metered Resource Type the Team may hold concurrently, set by the Capacity Planner. A safety valve against monopolizing a pool, not a rationing mechanism.

Inventory The record of every Resource that exists, its Resource Type, and who currently holds it or has it on Hold.

Requests

Request A single request, owned by a Team and submitted by its Manager, containing one or more Request Lines that may span all three Allocation Classes.

Request Line One need inside a Request: a Resource Type, a quantity, and a time window. Every line has exactly one Allocation Class, taken from its Resource Type.

Suggestion A single Request Line a Member proposes to their own Manager. Has no allocation effect until the Manager accepts it into a Request; several Suggestions may be accepted into the same line.

Required line / Optional line A Manager's marking on each Request Line. A Request is fulfilled only when every Required line is allocated; Optional lines allocate if they can and never block the outcome.

Hold A tentative, uncharged allocation of an Exclusive or Metered line placed while a Request's Matched lines await approval. Commits when the Request is fulfilled, releases when it fails or the Objection Window lapses.

Objection Window The bounded period a lending Manager has to approve or refuse a Matched line before the Request escalates to the Capacity Planner. Also the lifetime of any Hold on that Request.

Candidate A Person who satisfies a Matched line's skills and availability, has no Loan overlapping the line's window, and appears on its Shortlist. Becomes a Loan once their Manager approves.

Shortlist The list of Candidates the system produces for a skill-based Matched line, ordered by fewest person-days lent in the current Period, from which the requesting Manager selects.

Waitlist The ordered list of Required lines that could not be satisfied for a Resource Type and window. Priority lines first, then by submission time; a Bumped line goes to the head. Optional lines are never waitlisted.

Loan A Person allocated to a Team other than their own for a time window. The fulfilled form of a Matched line. A Person is in at most one Loan at a time.

Priority line An Exclusive Request Line that is charged a premium, a multiple of the Unit Price set by the Capacity Planner. Can Bump a standard line and cannot itself be Bumped.

Bump The displacement of a standard Exclusive line by a Priority line. The displaced line goes to the head of the waitlist with a full Refund. Not allowed inside the bump cutoff before the window starts.

Release A Manager giving back an allocated Exclusive or Metered line, in whole or in part, before it is consumed, or ending a Loan early. Triggers a Refund according to the Refund Schedule.

Credits

Credit The unit of account in the Ledger. No real money; used for showback, not billing.

Ledger The record of every Team's Account and every Transaction. Holds no capacity information.

Account A Team's standing in the Ledger: its granted credits, its earned credits, and its held credits.

Balance A Team's granted credits plus earned credits.

Available Balance Balance minus held credits. What a Team can commit to a new Request.

Held credits Credits set aside for a Request that is Pending, equal to what its Holds will cost when they commit.

Transaction One entry in the Ledger against an Account. Exactly four kinds: a Grant (from the Capacity Planner), a Charge (for an allocated Exclusive or Metered line), an Earning (for a Loan), or a Refund (for a Release).

Grant Credits given to a Team by the Capacity Planner at the start of a Period, or as a mid-period adjustment. Expire at the end of the Period.

Period The fixed calendar interval over which Grants are issued and expire. Set by the Capacity Planner.

Statement A Team's per-Period view of its Account: Grants, Charges, Earnings, Refunds, and what expired. The showback artifact.

Unit Price The base credits per unit of an Exclusive or Metered Resource Type (per hour of a room, per GPU hour), set by the Capacity Planner, before any Price Band multiplier. Matched lines have no Unit Price.

Price Band A time-of-day and day-of-week window, defined per Resource Type by the Capacity Planner, carrying a multiplier on the Unit Price. Rewards use at low-demand times.

Quote The credits a Request Line will cost, shown to the Manager while drafting and locked when the Request is submitted. The Charge that posts equals the Quote.

Day Rate The flat credits a Team earns per person-day of a Loan, set once by the Capacity Planner. The same for every Person regardless of skill.

Refund Schedule The rule, set per Resource Type by the Capacity Planner, giving the fraction of a Charge returned on Release as a step function of how far ahead of the window start the Release happens. Never reaches 100%.

Design terms

Module A unit of the design with one area of responsibility: one Java package under `edu.cwru.openbench`, or the web client. A back-end module owns its tables and exposes only its application services and the DTOs they take and return.

Application service A Spring bean that is a module's public entry point, such as `RequestLifecycleService`. Each method runs in one transaction unless marked read-only and carries the role check for its action.

Port An interface declared by a lower module and implemented by a higher one, so the lower module can ask a question without depending upward. The only port is `ReductionGuard`, declared by Inventory and implemented by Allocation.

Capability interface An optional interface an allocator implements beside `Allocator` when its Allocation Class has that behavior: `Waitlistable`, `Bumpable`, or `ApprovalDriven`. Requests asks the `AllocatorRegistry` whether a class has one.

Exclusion constraint A PostgreSQL constraint that rejects a row whose values conflict with an existing row under the given operators. It keeps Held or Allocated occurrences of one Resource from overlapping and Loans of one Person from overlapping.

C Inspection Report

This document was drafted on 2026-10-05 from the design brief and the architecture decision records. The same evening the four team members, J. Nana, B. Lujan, F. Turopov, and Y. Shnazbayev, ran a drafting review over the complete first build, with a reviewing agent, that checked every name against the brief, every requirement against Appendix A, and every figure against the text that cites it; it raised 42 issues, and the 21 that touched the design are the rows below, each credited to the member who raised it, all resolved in this version. Each member then reads the whole document and records further findings; the team meets, decides a resolution for each finding, and records its status.

One row per finding; categories: ambiguity, incompleteness, incorrectness, inconsistency, infeasibility, traceability, clarity; status Open, Closed (version), or Rejected (reason); locations name the full edition's sections.

Table 7: Inspection findings.

ID	Found by	Location	Category	Finding	Resolution	Status
I-1	J. Nana	Sec. 4.7, 4.9, 3.2.4, 5.3.4; Fig. 15	inconsistency	Lapse handling was defined two ways.	`lapsedWindows` returns Pending or Waiting Requests with an unescalated Outstanding approval; `onWindowLapse` is idempotent.	Closed (1.0)
I-2	B. Lujan	Table 6; Sec. 4.7; Fig. 12, 16	inconsistency	`requireAffordable` was said to lock the account row, yet S1 and S5 call it before `lockTypes`.	`requireAffordable` is an unlocked early refusal; `holdCredits` and `commitCharges` take the account lock after the type locks.	Closed (1.0)

Inspection findings (continued)

ID	Found by	Location	Category	Finding	Resolution	Status
I-3	F. Turopov	Sec. 4.7 vs 5.3.2; Fig. 13	inconsistency	The `allocate` step and scenario S2 ordered commits, Charges, and Loans differently.	S2 follows the Sec. 4.7 order: re-evaluate, drop Optional lines, commit, charge, form Loans, post Earnings.	Closed (1.0)
I-4	Y. Shnazbayev	Sec. 6.2; Table 12; Fig. 20	inconsistency	The hourly-sum query joined Requests' tables, against the dependency rule.	The query reads only Allocation tables.	Closed (1.0)
I-5	B. Lujan	Fig. 6, 20; Sec. 6.2	inconsistency	The Loan exclusion constraint excluded early-ended Loans, letting a new Loan overlap days actually lent.	Clause removed; `endLoanEarly` shortens `ends_on`, so the Loan stays in the constraint.	Closed (1.0)
I-6	J. Nana	Table 12; Fig. 20 vs Sec. 4.6	incompleteness	Ledger relied on `held_credit`, `period_close`, and `transaction.reason`, which the schema did not list.	Both tables and the column added to the table list and the schema figure.	Closed (1.0)
I-7	Y. Shnazbayev	Sec. 5.3.7; Fig. 18; Sec. 4.8	inconsistency	Rollover idempotency was keyed on `period_close` in Ledger but on a zero Grant in S7.	`period_close` is the key and holds the expired amount.	Closed (1.0)
I-8	F. Turopov	Fig. 12	incorrectness	S1's guards sent a Request needing approval but with one unsatisfiable Required line to Waiting instead of Pending (**Lifecycle.Evaluate**).	Guards corrected: unfit Required lines join the Waitlist while the Request enters Pending.	Closed (1.0)
I-9	J. Nana	Sec. 5.3.2; Fig. 13; Sec. 4.5	infeasibility	S2 had `MatchedAllocator.commit` report an ineligible Candidate and then `toWaiting` write in one transaction.	The re-check runs in `evaluateAll` before any commit; `commit` only guards.	Closed (1.0)
I-10	B. Lujan	Table 13; Sec. 8.4 vs Tables 4, 7	inconsistency	The API mapped a code `ILLEGAL_TRANSITION` that no module raises.	`WRONG_STATE` used; all codes describing the object's current state map to 409, the rest to 422.	Closed (1.0)
I-11	Y. Shnazbayev	Sec. 8.4 vs 4.5	inconsistency	An exclusion-constraint violation was answered 409 in one section and 500 in another.	Logged as a bug and answered 409 `ALLOCATION_CONFLICT` in both.	Closed (1.0)
I-12	F. Turopov	Sec. 4.1, 4.8, Table 6 vs Sec. 7, 8	inconsistency	Out-of-scope reads raised `ForbiddenException` in the modules but 404 in the API.	Out-of-scope reads raise `NotFoundException` (404).	Closed (1.0)
I-13	J. Nana	Sec. 4.7, Table 4 vs 5.1, Table 10, S6	inconsistency	Sec. 4.7 forbade a type lock after an account lock; the waitlist fill takes one.	The waitlist fill is named as the one such path, covered by the retry.	Closed (1.0)
I-14	B. Lujan	Table 8 vs Table 11	inconsistency	The Waiting-to-Pending guard differed between the two transition tables.	Both read: window running, approvals outstanding; unfit Required lines stay on the Waitlist.	Closed (1.0)
I-15	F. Turopov	Fig. 6; Sec. 5.2; Table 10	inconsistency	Escalated appeared both as an approval state and as a flag.	Escalated is a flag; states are OUTSTANDING, APPROVED, REFUSED, CANCELED everywhere.	Closed (1.0)
I-16	Y. Shnazbayev	Fig. 6; Table 10	inconsistency	Row states omitted RELEASED and EXPIRED; a refused Hold went to RELEASED.	States completed in the figure; refused Holds go HELD to EXPIRED.	Closed (1.0)
I-17	B. Lujan	Fig. 5, 6, 8	incompleteness	Class diagrams omitted nine operations the text relies on.	All added to the diagrams.	Closed (1.0)
I-18	J. Nana	Sec. 8.1 vs 4.1, 4.5, 4.7	ambiguity	Scoped reads took no `Actor`, which would refuse Allocation's and Inventory's own checks.	A request-scoped `CurrentActor` supplies scope; modules use the unscoped `MemberService.summary`.	Closed (1.0)
I-19	Y. Shnazbayev	Sec. 4.7	incompleteness	A rollover while a Request was Pending made `commitCharges` throw and roll back the approval.	`allocate` sends the Request to Waiting and keeps the approval.	Closed (1.0)
I-20	F. Turopov	Fig. 11	incompleteness	The line state machine lacked Held to Bumped and the system-refused approval.	Both transitions added.	Closed (1.0)
I-21	J. Nana	Sec. 5.1 vs Sec. 8.6	inconsistency	Sec. 5.1 says the ArchUnit test rejects an `isolation` attribute on `@Transactional`, but the Architecture test did not list it.	The check added to the Architecture test.	Closed (1.0)